SEDust: User Manual

Abstract

This manual describes the model-agnostic dust library under `SEDust/sed/`. It computes the infrared emission spectrum *and* the transport optics (extinction, scattering, asymmetry) of a dust mixture for an arbitrary local radiation field, and is intended to be linked into a Fortran 3D radiative-transfer (RT) code. It handles several dust models as peers — the HD23 *astrodust*+PAH model, the Draine & Li (2007) silicate+carbonaceous model, the Zubko, Dwek & Arendt (2004) BARE-GR-S model, and arbitrary file-defined models — through one object type, one emission call, and one extinction call. The numerically validated stochastic-heating solver is reused unchanged; the library is an additive layer on top of it.

This version is the **polarized** branch (v1.20). It is a *superset* of the scalar branch (v1.00): every scalar capability documented here is present there and computes the same numbers, and v1.20 adds the polarized cross sections, the direction-dependent extinction matrix and the aligned-grain scattering matrix (§6.). The one non-additive difference is that the `status` codes of `sed_init` / `build_astrodust` are numbered differently, because codes 3–5 are taken by the polarized inputs: what is `3` and `4` in v1.00 is `6` and `7` here (§4.5). Code that only tests `status /= 0` is unaffected.

Contents

1. Overview

A *dust model* is represented by a single derived type, `dust_model_t`. You build it once (per RT run), then call `dust_emission` once per RT cell with that cell's local mean intensity:

```
use dust_lib
type(dust_model_t) :: m
real(wp), allocatable :: J_lam(:), lamI_total(:)

call build_dust(m, 'astrodust', '../data') ! once
allocate(J_lam(m%NLAM), lamI_total(m%NLAM))
do icell = 1, ncells
   ! ... assemble J_lam(:) (mean intensity on the grid m%lam) for this cell ...
   call dust_emission(m, J_lam, lamI_total) ! per cell
   ! ... lamI_total(:) is lambda*I_lambda / N_H for the cell ...
end do
```

Design. The library wraps the existing, FIR-validated solver core (`sed_grain_loop`, `calc_Teq`, `calc_P`) without modifying it. A model is a set of grain *populations* (one per species and charge state), each carrying its own size distribution, absorption cross sections, enthalpy, and Planck-integral tables. `dust_emission` loops the populations through the solver and sums their emission into named output *channels*. The same populations carry the extinction-side optics, so `dust_extinction` (§5.) returns the model's opacity from that one object as well, and emission and extinction both reach the transfer code through calls on one model. Its scalar cross sections are served out of a size-integrated table the builder attached (§5.); the polarized ones are computed on every call, because they follow the host's runtime alignment state.

One active model at a time. The module-global grids (`lam`, `T_first`, ...) hold the *currently built* model's working set. Calling `build_*` makes that model active. This is sufficient for an RT run that uses a single dust model; to switch models, rebuild.

2. Building the library

From `SEDust/sed/`:

```
make libsedust.a # the library archive (link this into your RT code)
```

Nothing in it reaches the T-matrix, so `SEDust/tmatrix` need not be built for the default, non-ionizing Q table (1129 wavelengths, 0.0912 µm to 39 810 µm). The separately shipped `q_astrodust_P0.20_Fe0.00_1.400_euv.dat` has 1762 wavelengths and reaches 1×10^{-4} µm; explicit EUV commands select that file. What follows applies to a model that forms a band below the table it was given. Built from this archive, such a model *refuses* the spheroid (`euv_tmatrix`

= .true., the default whenever lam_min is given) with status = 11, rather than answering with a different particle behind the caller's back; euv_tmatrix = .false. asks for the volume-equivalent sphere explicitly.

A host that wants such a band on the spheroid of the Q table (§7.3) builds the T-matrix into the *same* archive, so the link line does not change:

```
cd SEDust/tmatrix && make libtmatrix.a # T-matrix
cd SEDust/sed && WITH_TMATRIX=1 make libsedust.a
```

and names the spheroid once, before the model is built:

```
use euv_astrodust_tmatrix, only: use_tmatrix_euv_band_optics
call use_tmatrix_euv_band_optics()
```

SEDust/sed/build_lib.sh is the same two builds as a standalone script, for an RT tree that carries its own copy of sed/src/; it writes sed/lib/ and takes WITH_TMATRIX=1 the same way. Of the CLI drivers, calc_sed.x astrodust and calc_kext.x make that call themselves — they are the two that would compute an astrodust EUV band if one ever formed.

The compiler is gfortran. The default flags are -O3 -ffree-line-length-none -fopenmp plus a warning set — deliberately *portable*, so that a libsedust.a built on one node runs on any other. A -march=native line is present but commented out; enabling it can change the last digits through FMA contraction. libsedust.a is a static archive of all library objects; the Fortran .mod files are written alongside it in SEDust/sed/. Because those .mod files share one directory, the Makefile carries .NOTPARALLEL and builds serially; make -j is not supported until the object and module graph lives in a separate directory.

3. Models and builders

Each builder fills a dust_model_t and defines the model's output channels (Table 1).

builder	model	channels	notes
build_astrodust	HD23 astrodust+PAH	AD, PAH	N_C=417, D16 graphite
build_dl07	Draine & Li 2007	SIL, CARB	N_C=470, D03 graphite
build_zubko	Zubko (ZDA 2004) BARE-GR-S	PAH, GRA, SIL	neutral PAH only
build_from_files	file-defined	CH1, CH2, ...	generic loader

Table 1: Built-in model builders. Each sets the model's own optics/abundance parameters internally.

Signatures (all optional arguments in brackets; every optional is a keyword argument in the order shown):

```
call build_astrodust(m, qtable_path, sizedist_path, NT, T_lo, T_hi &
                  [, status] [, qpol_path] [, qpol_wave_path] [, qpol_aeff_path] &
                  [, scatmat_path] [, load_polarized_optics] &
                  [, lam_min] [, astrodust_index_path] &
                  [, qpol_euv_path] [, qpol_euv_wave_path] &
```

```
                  [, kext_path] [, euv_tmatrix])
call build_dl07 (m, qtable_path, sizedist_path, sd_index, U, NT, T_lo, T_hi &
                  [, status] [, lam_min] [, kext_path] [, lam_axis] &
                  [, include_euv] [, stored_q_dir])
call build_zubko (m, config_path, data_dir, NT, T_lo, T_hi [, status] [, kext_path] &
                  [, lam_min] [, include_euv] [, qtable_path] [, optics])
call build_from_files(m, descriptor_path, data_dir, NT, T_lo, T_hi [, status] &
                  [, kext_path] [, lam_min] [, include_euv])
```

The polarized arguments (`qpol_*`, `scatmat_path`, `load_polarized_optics`) are described in §6.; the rest are common to both branches.

- `NT`, `T_lo`, `T_hi`: number and range [K] of the internal temperature grid (typical `200, 2.7, 5000`).
- `sd_index` (DL07): WD01 size-distribution index — `7` = MW R_V=3.1, $q_{\rm PAH}$=4.58%; `29/30/31` = LMC2; `32` = SMC.
- `U` (DL07): ISRF intensity scaling (the DL07 reference uses U=1).
- `data_dir` (Zubko / files): directory holding the data files, e.g. `data/zubko/`.
- `status` (optional, all builders): error report; see §4.5. When present, a missing or malformed input file is reported through it and the model is not built, so an RT host can recover. `build_dust` maps every builder's own stage number onto *one* vocabulary, so a host on the single entry point does not branch on the model name to read a code, and takes an optional `message` carrying the reason in words — what an MPI host prints before a collective abort.
- `data_dir` (`build_dust`): the data root for the whole library while the build runs. The dielectric functions, the default extinction curves and the stored cross-section tables all resolve inside it, so a host may call `build_dust` from any working directory with an absolute path, and a file that cannot be read arrives as a `status`, never as a runtime abort. A path the caller names itself is used as given and never composed with the root. A host that calls the four builders directly sets the root once with `sed_set_data_root`, re-exported from `dust_lib`.
- `stored_q_dir` (optional, DL07): where this model's stored cross sections come from. Omitted, the model's own directory under the data root, with the `/qtable` of an HDF5 `qtable_path` ahead of it. Passed blank, nothing stored is read and every optic is solved from the dielectric functions.
- `qtable_path` (optional, Zubko): the model's HDF5 product, which is where its stored cross sections then come from. This is an argument rather than hidden state so that a program writing `/kext` into a product takes its optics from the `/qtable` of that same product.
- `lam_min` (optional, astrodust and DL07): shortest wavelength [μm] the model must cover, for a host that transports ionizing radiation. See §7.. **Omitting it reproduces the previous behavior exactly** — the grid, and every

number derived from it, is unchanged. For astrodust what it does depends on the Q table: on the default 1129-wavelength one a floor below 0.0912 μm builds a band, while the `_euv` table already reaches the short end of the DH21 dielectric function, so there every value that is not refused falls inside the table and prepends nothing. DL07 can be carried to 6.205×10^{-5} μm below either, because its D03 optical constants reach past both.

- `astrodust_index_path` (optional, astrodust only): the dielectric function the model is made of. It is read to decide how far `lam_min` may reach, and it would supply the optics of a band below the Q table if one ever formed. It must be the file `qtable_path` was computed from — same porosity, iron fraction and axial ratio — or the model changes material at the seam. Omitted, it is the default that pairs with the shipped Q table:

```
data/dielectric/index_DH21Ad_P0.20_0.00_1.400 <->
data/astrodust/q_astrodust_P0.20_Fe0.00_1.400.dat
```

- `kext_path` (optional, all builders): the precomputed size-integrated extinction table this model serves through `dust_extinction` (§5.), loaded at build time. Naming a file that cannot be read *fails the build*; omitting it takes the model's default.

- `euv_tmatrix` (optional, astrodust only, default `.true.`): how the optics of a band below the Q table would be computed — `.true.` the T-matrix on the table's own $b/a = 1.400$ spheroid, `.false.` Mie on the volume-equivalent sphere (§7.3). Ignored when `lam_min` asks for no extension, which on the `_euv` Q table is every legal request; on the default 1129-wavelength table a floor below 0.0912 μm does reach it.

Two routes to the Zubko model. `build_zubko` evaluates the ZDA log-polynomial size-distribution *formula* (coefficients read from `ZDA_BARE_GR_S_Config.dat`) on the optics-table radii; `build_from_files` with `data/zubko/zubko_descriptor.txt` instead interpolates the *tabulated* `SzDist` files onto the same radii. The two shapes agree to $\sim 10^{-5}$, but the distributed tables carry a normalization that sits below $A\,g(a)$ of the config by 0.79% (PAH), 1.13% (graphite) and 0.41% (silicate), and they are sampled at only 24 / 62 / 63 radii, so interpolation near $a_{\max}$ adds a little more. Together this moves the size-integrated $C_{\rm ext}$ by up to 2.10% (median 1.02%). **The formula is the default and is faithful to the ZDA definition**; the tabulated route exists to reproduce the benchmark's own size grid.

The ZDA optics are a homogeneous-sphere Mie calculation, and that was checked. The three ZDA optics tables are the model definition, so nothing is recomputed at run time. What they *are*, however, is stated in two independent places and was verified here. Zubko et al. (2004, §3) write that for the bare silicate, graphite and amorphous-carbon particles "the extinction and absorption cross sections were calculated using Mie theory for spherical dust particles (Bohren & Huffman 1983)" — the effective-medium step of that paper applies to its *composite* models only. Each shipped file repeats it: the silicate and graphite headers name the dielectric function they were computed from (Draine's `eps_Sil` and `eps_Gra`), while `PAH_28_1201_neu.dat` names the Li & Draine (2001) and Draine & Li (2007) absorption cross sections instead — that component is not Mie and has no refractive index to be recomputed from.
Recomputing `suvSil_121_1201.dat` with the library's own Bohren–Huffman Mie on Draine's published `eps_Sil`, over all 121×1201 cells:

- at a fixed wavelength the ratio $Q^{\rm ours}/Q^{\rm table}$ is *independent of grain radius to four digits* across the whole 3.55×10^{-4}–$100\,\mu$m range (e.g. 1.0171 at $9.9\,\mu$m and 1.0067 at $99.8\,\mu$m at every radius);
- $\langle\cos\theta\rangle$ agrees to $\sim10^{-4}$.

Both are properties of the sphere and the size parameter, so they settle the question the comparison was asked: the table is a homogeneous-sphere Mie calculation and the library reproduces it. What is left is a purely wavelength-dependent factor — mean $|\Delta Q_{\rm abs}| = 0.59\%$ over 10–$100\,\mu$m, 1.7% over 1–$10\,\mu$m, rising at the extreme long-wavelength end — and a radius-independent, wavelength-dependent residual can only come from the refractive index, not from the scattering solution. The published `eps_Sil` is tabulated on a different wavelength grid from the table (200 per decade, ending at $10^3\,\mu$m, against 171 per decade to $10^4\,\mu$m), so the file the table's generator actually evaluated was a resampled and extended copy that is not the one distributed. Reproducing the tables to better than a percent would need that copy; nothing in SEDust depends on it, because the tables themselves are what every Zubko build reads.

4. Computing emission

4.1 Single-cell exact solve

```
call dust_emission(m, J_lam, lamI_total [, lamI_chan] [, status])
```

- `J_lam(m%NLAM)`: the local *mean intensity* on the grid `m%lam` [μm]. For a scaled Mathis ISRF use `call J_Mathis(U, m%lam, J_lam)` from module `radfield`.
- `lamI_total(m%NLAM)`: the summed SED, $\lambda I_\lambda/N_{\rm H}$ [erg s^{-1} cm^{-2} sr^{-1} H^{-1}] — the HD23 convention.
- `lamI_chan(m%NLAM, m%n_channel)` (optional): the SED of each channel.
- `status` (optional): error report; see §4.5.

4.2 Solver choice (`m%stoch_method`)

The field `m%stoch_method` selects how each grain is heated (Table 2); `dust_emission` honors it.

`m%stoch_method`	method	note
`'heuristic'`	GD look-ahead T-window narrowing	**default**; fastest, serial
`'draine'`	Guhathakurta & Draine (1989)	reference GD
`'qm'`	energy-space transition matrix (thermal-discrete)	most detailed
`'equil'`	single-grain equilibrium (no stochastic)	see §4.3

Table 2: Stochastic-heating solver options. The first three resolve the grain temperature *distribution* (PAH/NIR features); `'equil'` forces equilibrium.

4.3 Equilibrium-temperature options

Two options replace the full stochastic solve when an equilibrium approximation is wanted:

(1) Equilibrium temperature for each grain type and size. Set `m%stoch_method = 'equil'` and call `dust_emission`. Every grain is placed at its own equilibrium temperature $T_{\rm eq}$, computed from *that grain's* absorption cross section, and emits $C_{\rm abs}\,B_\lambda(T_{\rm eq})$. No stochastic excursions are computed.

(2) A single equilibrium temperature for the whole model.

```
call dust_emission_single_teq(m, J_lam, lamI_total [, Teq_out])
```

All dust shares one temperature, found from the type- and size-integrated (dn-weighted) total absorption cross section $C^{\rm tot}_{\rm abs}(\lambda) = \sum_{\rm pop}\sum_a {\rm d}n(a)\,C_{\rm abs}(\lambda,a)$. The single $T_{\rm eq}$ satisfies $\int C^{\rm tot}_{\rm abs} J\,{\rm d}\lambda = \int C^{\rm tot}_{\rm abs} B(T_{\rm eq})\,{\rm d}\lambda$, and the emission is $C^{\rm tot}_{\rm abs}\,B_\lambda(T_{\rm eq})$. The optional `Teq_out` returns that temperature.
Both options conserve energy (the emitted bolometric power equals the absorbed power); they differ from the stochastic solve only in the SED *shape* — a smooth modified blackbody rather than one carrying PAH/NIR features.

4.4 Precomputed table + interpolation

When the field in each cell is a fixed spectral *shape* scaled by an intensity U, precompute a table once and interpolate per cell:

```
type(dust_emis_table_t) :: tab
call dust_build_table(m, J_ref, U_grid, tab [, status]) ! once
call dust_emission_interp(tab, U, lamI_total [, lamI_chan] [, status]) ! per cell
```

`J_ref(m%NLAM)` is the reference field shape (at U=1) and `U_grid(:)` an ascending intensity grid. The table is built from exact solves, so it is exact at the grid points; interpolation is log–log in U. The stored emissivities are floored at 10^{-300} before the log, so a grid node whose true value is 0 comes back as 10^{-300} rather than exactly 0. Both calls take an optional final `status` (§4.5).

Caveat. Stochastic emission is a *nonlinear* functional of the full $J(\lambda)$, not of a scalar. The table is valid only when the cell field is $U\times$(fixed $J_{\rm ref}$). For cells whose field *shape* departs from $J_{\rm ref}$ (e.g. hardened spectra near hot stars), use the cell-by-cell exact `dust_emission`.

4.5 Optional error status

The builders and the emission/table calls each accept an optional final `status` argument (`integer, intent(out)`). When it is present, an invalid model, argument, or input file is reported through it (`status = 0` on success, nonzero on failure) and the process keeps running, so a 3D RT host can recover; when it is omitted, the same condition prints a message and `stop`s the run, which the CLI drivers rely on. For the builders the nonzero value only names the failing stage: the contract is simply `0` = built, nonzero = build failed.

`build_dust` carries a vocabulary of its own, and maps each builder's stage number onto it, so a host on the single entry point never branches on the model name to read a code. It also takes an optional `message` (`character, intent(out)`) carrying the reason in words, which is what an MPI host prints before a collective abort:

0	built	6	calorimetry
1	optics table	7	grid inconsistent between components
2	size distribution	8	EUV spheroid optics unavailable
3	dielectric function	9	model definition (config / descriptor)
4	`lam_min` not coverable	10	polarized optics
5	extinction table	90	model name not one of the four
		91	`from_files` without `config_path`

The four builders keep their own numbering, listed below, for a host that calls them directly.

- `dust_emission`: 1 unknown `stoch_method`; 2 `'qm'` selected but a population's radii are unset; 3 `m` is not the most recently built model. This routine solves through `sed_grain_loop`, which reads the module grids the last build filled, so it can answer for that model and no other; a stale model used to be answered with another model's numbers in silence. `dust_extinction` and `size_integrated_extinction` are *not* restricted — both read only the model argument.

- `dust_build_table`: 1 `size(J_ref)`$\neq$`m%NLAM`; 2 `size(U_grid)` < 2; 3 `U_grid` not positive and strictly increasing.

- `dust_emission_interp`: 1 `U` ≤ 0; 2 `size(lamI_total)`$\neq$`tab%NLAM`; 3 `lamI_chan` present but not (`tab%NLAM, tab%n_channel`).

- `build_dl07`: 1 Q-table load failed; 2 size-distribution load failed; 7 `lam_min` is shorter than the D03 dielectric functions ($6.1992 \times 10^{-5}\,\mu$m — the longer-reaching of astrosilicate and graphite), which is refused rather than served with a refractive index frozen at the table boundary. Codes 3–6, 8 and 9 cannot occur on this path: the DL07 model has no polarized optics and needs no separate EUV dielectric function, because its optics are D03 Mie at every wavelength. The numbering is shared with `build_astrodust` on purpose, so a host can decode one table.

- `build_astrodust` (and `sed_init`): 1 Q-table load failed; 2 size-distribution load failed; 3 an explicit `scatmat_path` failed to load; 4 an explicit `qpol_path` could not be read (an implicit default degrades gracefully instead); 5 `load_polarized_optics=.false.` combined with an explicit `qpol_*/scatmat_path` (a contradiction); 6 the astrodust dielectric function failed to load (raised only when `lam_min` asks for the EUV band); 7 `lam_min` is shorter than that dielectric function's own shortest wavelength ($1.00003 \times 10^{-4}\,\mu$m), refused for the same reason as above; 8 the polarized table's wavelength grid does not sit node for node on the long-wavelength end of the model's grid (a companion EUV table that is simply *missing* is not an error — that band degrades to the announced zero of §7.4); 9 the EUV part of the grid runs outside the wavelengths a supplied companion table covers (0.0124–0.0912 μm); 10 an explicitly named `kext_path` failed to load; 11 `euv_tmatrix = .true.`

but the spheroid optics of a band below the Q table are not available (unreachable with the shipped table, which leaves no such band). **Codes 6 and 7 are 3 and 4 in the scalar branch (v1.00)**, which has no polarized inputs to occupy 3–5.

- `build_zubko`: `1` config read; `2` fewer than 3 components; `3` a component's optics read; `4` grid inconsistent; `5` a component's calorimetry read failed; `8` `optics` is neither `'zda'` nor `'mie_d03'`; `6` an explicitly named `kext_path` failed to load; `7` `lam_min` is shorter than this model's own optics table, which it cannot extend.
- `build_from_files`: `1` descriptor open; `2` too many `pop:` lines; `3` invalid channel; `4` no `pop:` lines; `5` optics read; `6` grid inconsistent; `7` size-distribution read; `8` calorimetry read failed; `9` an explicitly named `kext_path` failed to load; `10` `lam_min` is shorter than the model's own optics tables.
- **An explicitly named `kext_path` that fails to load** (§5.) is `5` in `build_dl07`, `6` in `build_zubko`, `9` in `build_from_files` — the next free code in each — and `10` in `build_astrodust`, whose polarized codes already occupy 3, 4, 5, 8 and 9. The scalar branch (v1.00) uses the same numbers except that its `build_astrodust` has no polarized codes and so uses `5` there too.
- `dust_extinction`: `1` an output array is not of size `m%NLAM`; `2` no extinction table was loaded for this model; `3` `m%lam` runs outside the table.

5. Computing extinction

`dust_extinction` is the extinction counterpart of `dust_emission`: a transfer code takes its opacity from the same model object, and on the same wavelength grid `m%lam`, as its emission.

```
call dust_extinction(m, Cext, Cabs, Csca [, gbar] [, Cpol_ext] [, Cbir_ext] &
                  [, albedo] [, status])
```

- `Cext`, `Cabs`, `Csca` (`m%NLAM`): extinction, absorption and scattering cross sections per H atom [cm^2/H], with $C_{\rm ext} = C_{\rm abs} + C_{\rm sca}$.
- `gbar(m%NLAM)` (optional): the scattering asymmetry $\langle \cos\theta \rangle$.
- `Cpol_ext(m%NLAM)` (optional): the dichroic (polarized) extinction [cm^2/H]; see §6..
- `Cbir_ext(m%NLAM)` (optional): the birefringent extinction [cm^2/H], the $f_{\rm align}$-weighted size integral of the orientation-resolved table's 4th block. It is 0 when the loaded table carries no birefringence (the release table does not); see §6..
- `albedo(m%NLAM)` (optional): $C_{\rm sca}/C_{\rm ext}$, and 0 where $C_{\rm ext}$ underflows to zero. It is derived here rather than by the caller so that every host gets the same convention at those wavelengths.
- `status` (optional): error report; see §4.5. It is 0 on success, 1 if an output array is not of size `m%NLAM`, 2 if no extinction table was loaded for this model, and 3 if `m%lam` runs outside the table.

Where the numbers come from. The four *scalar* outputs — `Cext`, `Cabs`, `Csca`, `gbar` — are read from a precomputed size-integrated extinction table, one of the `data/<model>/kext_*.dat` products of `calc_kext.x` (§8.), attached to the model at build time and interpolated onto `m%lam`. The transport optics of a dust model do not depend on the transport, so there is nothing to gain by repeating the size integral during a transfer run: the table *is* that integral, done once and recorded.

The two *polarized* outputs — `Cpol_ext` and `Cbir_ext` — are **not** tabulated. They are computed from the model's own orientation-resolved optics on every call, because their $f_{\rm align}(a)$ weight is runtime state a transfer code resets cell by cell through `dust_set_alignment` (§6.3), which a table fixed at build time could not follow. This asymmetry — scalar optics from a table, polarized optics from the model — is deliberate.

The first-principles size integral remains available under its own name and with an identical argument list:

```
call size_integrated_extinction(m, Cext, Cabs, Csca [, gbar] [, Cpol_ext] &
                                [, Cbir_ext] [, albedo] [, status])
```

It needs no table and computes all six outputs from the model's optics. It is what `calc_kext.x` calls to write the tables in the first place, and what the in-tree checks (`test_scalar_mode.x`, `test_euv_extension.x`) call when they have to compare one build of a model against another. Each output of it is the plain binned sum over every population of the model, since $\mathrm{d}n(a)$ already carries the bin width:

$$C_{\rm abs}(\lambda) = \sum_{\rm pop}\sum_{a} \mathrm{d}n(a)\,C_{\rm abs}(\lambda,a), \qquad C_{\rm sca}(\lambda) = \sum_{\rm pop}\sum_{a} \mathrm{d}n(a)\,C_{\rm sca}(\lambda,a),$$

$$\langle\cos\theta\rangle = \frac{\sum \mathrm{d}n\,C_{\rm sca}\,g}{\sum \mathrm{d}n\,C_{\rm sca}}, \qquad C_{\rm pol,ext}(\lambda) = \sum_{\rm pop}\sum_{a} \mathrm{d}n(a)\,C_{\rm pol,ext}(\lambda,a)\,f_{\rm align}(a). \tag{1}$$

A population that carries no scattering or polarized optics contributes zero to those terms. In the astrodust model the PAHs are exactly that case: they enter through absorption only, and the astrodust grains carry all the scattering and the asymmetry.

Dust mass per H, and mass opacity. Both routines return cross sections *per H nucleon*. A host that carries a dust mass density instead of a hydrogen column converts them with one number, which the model itself reports:

```
Mdust_H = dust_mass_per_H(m) ! [g/H]
kappa = Cabs / Mdust_H ! [cm^2 per gram of dust]
```

It is the model's own size distribution weighted by the solid density of each population's material,

$$\frac{M_{\rm dust}}{N_{\rm H}} = \sum_{\rm pop} \rho_{\rm bulk} \sum_{a} \tfrac{4}{3}\pi a^3\,\mathrm{d}n_{\rm pop}(a), \tag{2}$$

with a in cm and $\mathrm{d}n$ already carrying the bin width, so the opacity it produces refers to the same grains the emission does. Being a property of the model it is independent of wavelength and temperature: evaluate it once. A population whose builder states no density ($\rho_{\rm bulk} = 0$) is skipped rather than given a guessed one, and a model in which no population states a density returns 0.

The densities are part of each model's definition, not a free choice — the size distributions were normalized to element abundances with them:

model	population	$\rho_{\rm bulk}$ [g cm^{-3}]
astrodust	astrodust grains	2.74 (HD23 §2.3.2, already $\times(1-P)$)
	PAH	2.0 (HD23 $N_{\rm C} = 417\,(a/{\rm nm})^3$)
DL07	astrosilicate	3.5 (DL07 §2)
	carbonaceous	2.2 (graphitic carbon, DL07 §2)
Zubko, file-defined	each component	declared in its own optics table

Both DL07 values are the ones its own paper states, in the §2 paragraph that defines the effective radius $a = (3M/4\pi\rho)^{1/3}$: "amorphous silicate is assumed to have a mass density $\rho = 3.5\,{\rm g\,cm^{-3}}$, and carbonaceous grains are assumed to have a mass density due to graphitic carbon alone of $\rho = 2.2\,{\rm g\,cm^{-3}}$". The graphite value is deliberately *not* the 2.24 of WD01/LD01, which is the density implied by the carbon-atom count $N_{\rm C} = 470\,(a/{\rm nm})^3$ this builder selects; DL07 kept that count and quoted the rounder density, and the model built here is DL07's.

The DL07 carbonaceous grains are one continuous material sequence — PAH-like when small, graphite-like when large, joined by the DL07 $\xi(a)$ weight — with no size at which the solid changes, so the graphite density is used across the whole sequence rather than split at some radius. That the astrodust model's PAHs use 2.0 instead is HD23's convention for a different material in a different model, not a disagreement about graphite.

The same number normalizes the $K_{\rm abs}$ column of every `data/<model>/kext_*.dat` product (§8.), so a host that reads the tables and a host that links the library get the identical conversion. The values are 1.184949×10^{-26} g/H for astrodust, 1.750186×10^{-26} g/H for DL07 and 1.443932×10^{-26} g/H for Zubko BARE-GR-S.

Which DL07 normalization is followed, and why it matters. DL07's own Table 3 gives $M_{\rm dust}/M_{\rm H} = 0.0104$ for model $j_M = 7$ (MW3.1_60) — the very size distribution `build_dl07` selects — and the densities above reproduce it to about half a percent (1.750186×10^{-26} g/H, i.e. $M_{\rm dust}/M_{\rm H} = 0.01046$ with $m_{\rm H} = 1.6737 \times 10^{-24}$ g, +0.55%). Draine's tabulated optics for the same model, `kext_albedo_WD_MW_3.1_60_D03.all_2003`, state $M_{\rm dust}/N_{\rm H} = 1.870 \times 10^{-26}$ g/H in the file header, i.e. $M_{\rm dust}/M_{\rm H} = 0.0112$, which is 7% away from that author's own Table 3. The same file renormalizes the silicate distribution by 0.93 where the paper's Fig. 11 caption says 0.92.

SEDust takes the *optics* from the file — reproducing it is the point, and our $C_{\rm ext}$/H agrees with it to 0.1–0.3% over its whole range — and the *mass* from the paper. The two normalizations differ, so about 1% remains between our $K_{\rm abs}$ column and the file's; that residual is on Draine's side, not a free parameter here.

Choosing the table. Every builder takes an optional `kext_path` naming the file. Omitting it takes that model's default:

builder	default table
`build_astrodust`	`../data/astrodust/kext_astrodust_MW_euv.dat`
`build_dl07`	`../data/dl07/kext_dl07_MW_euv.dat`
`build_zubko`	`../data/zubko/kext_zubko_BARE_GR_S_euv.dat`
`build_from_files`	none

Every default is the EUV product, because it is the *widest* grid that model has: one file then serves a host that transports ionizing radiation and a host that does not, since the latter's grid is a subset of the former's, on the same nodes. The astrodust pair differs by the 633 wavelengths the `_euv` Q table carries below the Lyman limit, and `kext_astrodust_MW.dat` is also written through narrower field widths (§8.); both carry the dichroic column. The DL07 pair differs by those 633 and by the further 61 points its D03 optical constants reach past the table. The Zubko pair differs by 335: its own ZDA optics tables carry 336 nodes below the Lyman limit and the narrow product keeps the last of them, 0.089 98 μm, so that it still covers the band a host transports — there the wide product is the tables' own range and the narrow one is that range cut, the reverse of the other two. Naming a `kext_path` is otherwise how you point at a product of your own. A file-defined model has no default at all: its product is named after the model, which only the descriptor knows, so a host that wants extinction out of such a model must name the file.

A `kext_path` that cannot be read *fails the build* (§4.5 for the code, which differs by builder) — a host naming a file that is not there is a configuration error. A *default* that cannot be read does not fail the build: the emission-only drivers must still run, and `calc_kext.x` builds a model precisely in order to write the table that is not there yet. In that case `m%kext_n` is 0 and `dust_extinction` reports status 2.

The table is read at *build* time, not at query time, because these paths are relative to `sed/`. A host that changes directory around the `build_*` call — which is what MoCafe does — would otherwise find them broken by the time it asks for opacity.

Interpolation, and when it is exact. `Cext`, `Cabs` and `Csca` are interpolated linearly in $\log\lambda$–$\log C$; `gbar` linearly in $\log\lambda$, because it is signed and changes sign in the far infrared of some models. A wavelength of `m%lam` that coincides with a table node (within 10^{-6} relative, the tolerance that absorbs the round trip of λ through a decimal file) takes the tabulated value *unchanged*. Nothing is extrapolated: a grid running outside the table is refused with status 3, not served a frozen boundary value.

An astrodust or DL07 model therefore gets its numbers back bit-for-bit: its wavelengths are the T-matrix Q-table grid, and the default `_euv` kext table was written on the superset of it, so every model wavelength coincides with a table node and nothing is interpolated. The case that still interpolates is a model built with a `lam_min` below the table it was given, whose prepended points are log-spaced from that floor and so do *not* share the nodes of the default kext table; a host that needs that band at full fidelity should generate a table at its own `lam_min` (`./calc_kext.x dl07 <lam_min>`) and pass it as `kext_path`. On the `_euv` Q table astrodust cannot reach that case at all: no legal `lam_min` prepends anything (§7.).

Every model scatters. All four builders populate the extinction-side scattering optics, so `albedo` and `gbar` are physical for every model:

- **astrodust**: $C_{\rm sca}$ and $\langle\cos\theta\rangle$ from the random-orientation T-matrix Q table, at every wavelength of the model.
- **DL07**: Mie on the D03 astrosilicate and graphite dielectric functions (`q_silicate_full` / `q_graphite_full`), on the same random-orientation average ($\frac{1}{3}E\parallel c+\frac{2}{3}E\perp c$) as the absorption. The two carbonaceous charge states share one scattering description: charge shifts PAH absorption features, not the graphite sphere's scattering.

- **Zubko / file-defined**: the $Q_{\rm sca}$ and g columns of the model's own ZDA optics tables, read from the same file as $Q_{\rm abs}$. Those tables are a *homogeneous-sphere* Mie calculation — Zubko et al. (2004, §3) state it for the bare grains, and each file's header names the dielectric function it came from, Draine's `eps_Sil` and `eps_Gra`; the effective-medium step of that paper applies to its composite models, not to BARE-GR-S. The PAH file is not Mie at all: its header names the Li & Draine (2001) / Draine & Li (2007) absorption cross sections. §3. records what recomputing the silicate table with our own Mie gives.

A population that genuinely does not scatter leaves its scattering optics unallocated and contributes zero. In the astrodust model the PAHs are exactly that case — PAH scattering is Rayleigh-negligible — so they enter extinction through absorption only, and the astrodust grains carry all the scattering and the asymmetry.

Polarized optics are astrodust only. `Cpol_ext` and `Cbir_ext` come back as zeros from `build_dl07`, `build_zubko` and `build_from_files`, which carry no polarized optics at all, and from an astrodust model built with `load_polarized_optics = .false.` `dust_has_polarized_optics(m)` distinguishes the two cases.

6. Polarized dust optics

The library exports two polarized quantities for the HD23 astrodust model: a polarized *emission* spectrum through `dust_emission`, and a polarized *extinction* cross section through `dust_extinction` or, equivalently, through the precomputed extinction table. Both are *intrinsic*: the size integral and the alignment weighting are done, but the geometry of the local magnetic field is not applied. What remains for the calling code is set out in §6.6.
This section documents the interface only. For the alignment physics, the polarized transfer equation, and the sign and index conventions behind these quantities, see the companion report `aligned_grain_polarization.pdf` (§3 for the optical properties, §4 for alignment, §5 for transfer).

Wavelength coverage. `Cpol`, `Cpol_ext` and `Cbir_ext` come from an orientation-resolved table on 1129 wavelengths, 0.0912–39810 μm, so nothing has to be regenerated to use them anywhere in that range. That is the whole grid of a model built on the default Q table, whose 1129 wavelengths are the same ones; it is the *long-wavelength* part of a model built on the `_euv` table, because the scalar Q table was carried into the ionizing band and the orientation-resolved one was not, which leaves that model's 633 shortest wavelengths without polarized entries (§7.4). The *default* table is the HD23 release file `data/dielectric/q_DH21Ad_P0.20_Fe0.00_1.400.dat.gz`; SEDust's own regenerated table, `tmatrix/output/q_astrodust_jori_P0.20_Fe0.00_1.400.dat.gz`, is opt-in through `qpol_path` and is the one that carries the 4th (birefringence) block, so `Cbir_ext` is zero with the default and nonzero with it (§6.4). The angle-resolved *scattering* matrix is a separate, coarser product — five optical bands (§6.5) — and shortward of 0.0912 μm the polarized extinction has an announced hole (§7.4).

6.1 Polarized emission

```
call dust_emission(m, J_lam, lamI_total [, lamI_chan] [, status] [, lamI_pol])
```

- `lamI_pol(m%NLAM)` (optional): the intrinsic polarized emission, with the same shape and the same units as `lamI_total`. It is the emission a population of perfectly aligned grains would radiate with their symmetry axes in the plane of the sky.
- Only populations that carry both polarized optics and an alignment efficiency contribute. PAHs are taken to be unaligned by HD23 and add zero, as does any model built without polarized optics.
- The argument is optional and comes last, so every existing call site remains valid.

The polarized channel is accumulated at every point where the total emission is accumulated, using the same grain temperature weights, so a stochastically heated grain contributes to both consistently under all four values of `m%stoch_method`.

6.2 Polarized extinction

Two routes give the same quantity. The direct one is the optional `Cpol_ext` argument of `dust_extinction` (§5.),

```
call dust_extinction(m, Cext, Cabs, Csca, gbar, Cpol_ext)
```

which returns $\sum_a \mathrm{d}n_{\mathrm{Ad}}(a)\, C_{\mathrm{pol,ext}}(\lambda, a)\, f_{\mathrm{align}}(a)$ [cm^2/H] on `m%lam`. Prefer it in a code that already holds a model object. Unlike the four scalar outputs of that call, this one is computed from the model's optics rather than read from a table, so it follows the alignment efficiency the host has set (§6.3) — a tabulated column could not.

The second route is the file `data/astrodust/kext_astrodust_MW.dat`, written by `calc_kext.x astrodust`, see §8., which carries the dichroic extinction as an eighth column (Table 3). It is the right choice for a tool that wants the tabulated optics without linking the library, and it is valid only for the HD23 alignment fit it was written under. Codes that read only the first seven columns are unaffected by its presence, and `dust_extinction` is one of them: it reads no polarized column from any table.

6.3 Changing the alignment efficiency

By default the alignment efficiency is the HD23 fit. Two setters, both re-exported by `dust_lib`, replace it on a model that already exists.

```
call dust_set_alignment(m, f_max, a_align, alpha_align [, status])
call dust_set_alignment_profile(m, aeff_in, falign_in [, status])
```

The first installs the HD23 power-law rolloff

$$f_{\mathrm{align}}(a) \;=\; \frac{f_{\max}}{1+(a_{\mathrm{align}}/a)^{\alpha_{\mathrm{align}}}}, \tag{3}$$

with a_{align} in μm. The second installs an arbitrary tabulated profile: the caller supplies (`aeff_in`[μm], `falign_in`) pairs and each population's efficiency is interpolated from them onto its own radius grid, linearly in $\log a$ and clamped at the ends of the table. This is the route for prescriptions Eq. (3) cannot express: a RAT-derived Rayleigh reduction factor $R(a)$, or the GRADE-POL exponential $f_{\max}[1-\exp(-(0.5a/a_{\mathrm{align}})^3)]$.

col.	quantity	unit	note
1	lambda	μm	wavelength grid
2	albedo		$C_{\rm sca}/C_{\rm ext}$
3	<cos>		scattering asymmetry g
4	C_ext/H	cm^2/H	total extinction
5	C_abs/H	cm^2/H	total absorption
6	C_sca/H	cm^2/H	total scattering
7	K_abs	cm^2/g	mass absorption coefficient
8	C_polext/H	cm^2/H	dichroic extinction (new)

Table 3: Columns of kext_astrodust_MW.dat. Columns 1–7 are written for every model; column 7 is $C_{\rm abs}/{\rm H}$ divided by that model's dust mass per H, Eq. (2). Column 8 is $\sum_a {\rm d}n_{\rm Ad}(a) C_{\rm pol,ext}(\lambda, a)\, f_{\rm align}(a)$, the maximum polarized extinction, quoted before any geometric factor, and this astrodust is the only model with polarized optics, so its two products are the only ones that carry it. On the 1129-wavelength product every row of column 8 is a measured value, because those wavelengths ARE the orientation-resolved table's. On the _euv product the 633 rows below $0.0912\,\mu$m are exactly zero, and the header says in as many words that this is the announced deficit of §7.4 rather than a measurement.

Both refill falign only for populations that carry polarized optics. PAHs, which HD23 take to be unaligned, are left untouched and keep contributing zero.

No temperature solve is repeated. This is the property worth knowing before designing a parameter study. The alignment efficiency enters as a weight in the size integral, *outside* the temperature solution: it multiplies the polarized accumulator and never enters $P(T)$. Changing the alignment therefore does not invalidate any temperature distribution, and lamI_total is bit-for-bit unchanged across a setter call. A scan over alignment parameters costs one dust_emission call per point, not one model build.

Negative efficiencies are allowed on purpose. dust_set_alignment_profile requires falign_in to lie in $[-1, 1]$ and *rejects* rather than clamps a value outside it, since $|f| \leq 1$ is the physical bound on a Rayleigh reduction factor and a value beyond it is a caller error that silent clamping would hide. Negative values inside the range are accepted: a grain with slow internal relaxation has $Q_X = -0.1$, a negative reduction factor that flips the polarization direction by 90°. That is the accepted origin of polarization parallel to $\vec{B}$ at millimeter wavelengths and perpendicular to it in the submillimeter, and clamping at zero would silently delete it. dust_set_alignment keeps $f_{\rm max}$ in $[0, 1]$, the power law having no such sign freedom.

With status present a bad argument sets it nonzero and returns; without it, the run stops. The codes are 1, 2, 3 in the order the arguments are listed above for dust_set_alignment ($a_{\rm align} \leq 0$, $\alpha_{\rm align} \leq 0$, $f_{\rm max}$ outside $[0, 1]$), and for dust_set_alignment_profile 1 for mismatched or too-short arrays, 2 for an aeff_in that is not positive and strictly increasing, and 3 for a falign_in value outside $[-1, 1]$.

Status code 4: a loaded aligned scattering table is now stale. If an aligned scattering table has been loaded (§6.5) and the requested profile differs from the one that table was integrated under, either setter returns the new code `4`. This is *non-fatal*: the $f_{\rm align}$ update completes and the emission and `Cpol_ext` channels re-integrate live under the new profile, so those outputs always honor the latest alignment. The code only signals that the loaded scattering matrices no longer correspond to it, because they were size-integrated once under the recorded profile. The sanctioned runtime variation of the aligned scattering optics is the scalar η (§6.5); a genuinely different profile requires regenerating the table with `run_scatmat_aligned.x profile=FILE`. A tabulated profile never matches the recorded analytic one and so always raises the flag.

6.4 Where the polarized optics come from

The orientation-resolved spheroid cross sections behind `lamI_pol` and `Cpol_ext` default to the Draine & Hensley (2021) release table. They can instead be taken from a table SEDust regenerates from the astrodust dielectric function with its own T-matrix engine: `build_astrodust` and `sed_init` accept an optional `qpol_path` (with `qpol_wave_path` and `qpol_aeff_path` for its grid axes) that overrides the default file. The regenerated table matches the release to a few parts in 10^4 where grains carry weight and is written in the release table's own format, so nothing else in the call changes. Its one deliberate benefit is that it supplies the polarized cross sections from the same exact optics as the total intensity, removing the mixed averaging convention the release forces; see `sedust_polarization_implementation.pdf` §7 and `tmatrix/run_q_jori.x`. Leaving `qpol_path` unset keeps the release table and the existing results, though an explicit `qpol_path` that cannot be read now fails the build with `status = 4` (and an unconditional stderr message), whereas the implicit default still degrades gracefully to an unpolarized model.

Scalar-only builds. `build_astrodust` and `sed_init` take an optional `load_polarized_optics`. Absent or `.true.` is the present behavior. `.false.` never opens the orientation-resolved polarized Q table: `Cpol`, `Cpol_ext`, `Cbir_ext` and `falign` stay zero, and the scalar `Cext`/`Cabs`/`Csca`/`gbar` and the total SED come back bit-identical to a polarized build at zero alignment. Combining it with an explicit `qpol_*` or `scatmat_path` is a contradiction and returns `status = 5`. The query `dust_has_polarized_optics(m)` reports whether the built model carries polarized optics at all; the aligned-scattering table's load state is separately visible as `scm_loaded`.

6.5 Aligned-grain scattering optics

For the astrodust spheroid SEDust also supplies the angle-resolved scattering optics of the aligned population, for a polarized RT code that transports a Stokes vector: the fixed-orientation Mueller matrix, the 4×4 extinction matrix, and the random-orientation remainder. These come from a table written by `tmatrix/run_scatmat_aligned.x` (five optical bands ship with SEDust); how it is built is documented in `sedust_polarization_implementation.pdf` §8. The API follows the library's own lifecycle — initialize once, then query along the photon path — split into two strictly separated layers.

Loading (once per run, serial). Pass the table path as an optional `scatmat_path` to `build_dust`, `sed_init` or `build_astrodust`, exactly like `qpol_path`. It reads `/polarized/scatmat_aligned` of the model's HDF5 product, or the ASCII table the T-matrix driver writes — the suffix picks the route:

```
call build_dust(m, 'astrodust', '../data', status=st, &
              scatmat_path='../data/astrodust/sedust_astrodust.h5')
```

The production table holds five optical bands — 0.36, 0.44, 0.55, 0.64 and 0.79 μm, approximately *UBVRI* — on a grid of 19 incidence angles θ_i × 181 scattering angles θ_s × 37 azimuths ϕ. Five bands is a deliberate choice, not a limitation of the format: polarized transfer is almost never wanted at every wavelength, and what matters is that any other wavelength *can* be produced (§8.6 — note the caveats there, since the scattering table is the less flexible of the two products). At run time the queries are a trilinear interpolation on that grid, a few hundred floating-point operations per scattering event, and `scatmat_band` maps a requested wavelength onto a stored band once.

The table is parsed and its size integrals kept in memory (about 81.5 MB for the production grid, growing linearly with the number of bands). A `scatmat_path` that fails to load is an error (`status = 3` from the builder), because a host that asked for the table depends on it. A gzipped table — both this and the polarized Q table ship `.gz` — is decompressed to a scratch copy under `TMPDIR` (else `/tmp`) named with the process id (`scatmat_aligned_<pid>.dat`, `q_jori_<pid>.dat`) and deleted after the read, so concurrent MPI ranks never collide and a read-only launch directory is never written. This layer is not thread-safe; call it from serial code before the photon loop.

Path queries (per event, thread-safe). Six routines, all re-exported by `dust_lib`, are pure reads of the loaded arrays: they write only their own output arguments, do no I/O and no allocation, and are safe to call concurrently from OpenMP photon threads. The whole cell dependence enters through two scalars the host already holds — the alignment scale η, and the incidence angle $\theta_i = \arccos(\hat{k}\cdot\hat{B})$ between the photon direction and the local field, from one dot product.

```
call scatmat_band(lambda_um, iband, exact) ! pick the band once
call extinction_matrix_aligned(iband, theta_i, eta, kmat) ! 4x4 [um^2/H]
call mueller_matrix_total(iband, theta_i, theta_s, phi, eta, z) ! 4x4 [um^2 sr^-1/H], absolute
call mueller_matrix_aligned(iband, theta_i, theta_s, phi, z) ! 4x4 [um^2 sr^-1/H], aligned only, eta=1
call mueller_matrix_random(iband, big_theta, f_tot, f_ref) ! 6-elt random matrices
call scattering_cross_sections(iband, theta_i, eta, csca_aligned, csca_unaligned)
```

- `scatmat_band`: `lambda_um` [μm] in; `iband` the nearest stored band; `exact .true.` if it matched to 10^{-3} fractionally. The hot path then takes `iband` so no wavelength search runs each event.
- `extinction_matrix_aligned`: `kmat(4,4)` [μm^2/H], the η-scaled aligned extinction matrix at incidence `theta_i` [deg]. Diagonal $\eta C_{\rm ext,al}$; $K(1,2) = K(2,1) = \eta C_{\rm pol,al}$ (dichroism); $K(3,4) = \eta C_{\rm bir,al}$, $K(4,3) = -\eta C_{\rm bir,al}$ (birefringence). `theta_i` in $[0, 180°]$ is folded to $[0, 90°]$, K being even under the equatorial reflection.
- `mueller_matrix_total` (**recommended**): the *absolute* combined phase matrix `z(4,4)` [μm^2 sr^{-1}/H] at grain-frame geometry and alignment η, with both populations already assembled,

$$z = \eta Z_{\rm al} + L(\pi-\sigma_2)\,F_{\rm unal}(\Theta)\,L(-\sigma_1)\,/\,(4\pi), \quad F_{\rm unal} = C_{\rm sca,tot}F_{\rm tot} - \eta C_{\rm sca,ref}F_{\rm ref},$$

with $\cos\Theta = \cos\theta_i\cos\theta_s + \sin\theta_i\sin\theta_s\cos\phi$ and the meridional rotation angles (closed forms, poles included)

$$\sigma_1 = \mathrm{atan2}(\sin\theta_s\sin\phi,\ \cos\theta_i\sin\theta_s\cos\phi - \sin\theta_i\cos\theta_s),$$
$$\sigma_2 = \mathrm{atan2}(-\sin\theta_i\sin\phi,\ \cos\theta_i\sin\theta_s - \sin\theta_i\cos\theta_s\cos\phi).$$

L is the Stokes rotation. Prefer this call to reconstructing the mixture from the public arrays: it is the one place the normalization and the rotation signs are certified.

- `mueller_matrix_aligned`: the aligned matrix alone, `z(4,4)` [μm^2 sr^{-1}/H] at the reference alignment ($\eta = 1$; scale it yourself). `theta_i`, `theta_s` in $[0, 180°]$, `phi` in $[0, 360°]$; the stored octants ($\theta_i \le 90°$, $\phi \le 180°$) and the rest are handled by interpolation plus the header's symmetry relations.

- `mueller_matrix_random`: `f_tot(6)`, `f_ref(6)`, the two six-element $(F_{11}, F_{22}, F_{33}, F_{44}, F_{12}, F_{34})$ random-orientation matrices at `big_theta` [deg], α_1-normalized as stored ($\frac{1}{2}\int F_{11}\,\mathrm{d}\cos\Theta = 1$, i.e. $\int F_{11}\,\mathrm{d}\Omega = 4\pi$). The absolute differential matrix is $C_{\rm sca}F/(4\pi)$: restore μm^2 sr^{-1}/H with `f_tot*scm_csca_tot(iband)/(4*PI)` and `f_ref*scm_csca_ref(iband)/(4*PI)`, the $1/(4\pi)$ turning the 4π-normalized F into the matrix whose element 11 integrates to $C_{\rm sca}$ over the sphere. (Omitting it overweights the remainder by 4π relative to $Z_{\rm al}$, whose stored matrix already integrates to $C_{\rm sca,al}$.)

- `scattering_cross_sections`: `csca_aligned` $= \eta C_{\rm sca,al}(\theta_i)$ and `csca_unaligned` $= C_{\rm sca,tot} - \eta C_{\rm sca,ref}$ [μm^2/H], for the albedo and the population split. An optional fourth output `csca_pol_aligned` $= \eta C_{\rm sca,pol,al}(\theta_i)$ is the polarized scattering cross section $\int Z_{12}\,\mathrm{d}\Omega$ of the aligned matrix (Peest et al. 2023), which the polarization-dependent albedo $\Lambda = [C_{\rm sca} + C_{\rm sca,pol}\,Q/I]/[C_{\rm ext} + C_{\rm pol}\,Q/I]$ needs; the random remainder contributes zero to it by azimuthal symmetry.

The η contract, and unit note. A cell's alignment enters only as η: the aligned matrix and the aligned K scale as η, and the unaligned remainder scattering matrix is $Z_{\rm unal} = [C_{\rm sca,tot}F_{\rm tot} - \eta C_{\rm sca,ref}F_{\rm ref}]/(4\pi)$ in absolute μm^2 sr^{-1}/H, whose element-11 solid-angle integral is $C_{\rm sca,tot} - \eta C_{\rm sca,ref}$. For the total extinction matrix the aligned population must not be counted twice: the isotropic `Cext` from `dust_extinction` already contains its reference-orientation average $C_{\rm ext,ref}$, so assemble

$$K_{\rm total} = (C_{\rm ext}^{\rm iso} - \eta C_{\rm ext,ref})\,I + \eta K_{\rm al}(\theta_i), \tag{4}$$

which returns $C_{\rm ext}^{\rm iso}$ on the diagonal at the incidence average and adds the direction-resolved dichroism and birefringence. These aligned-scattering quantities are in μm^2 (per H), whereas `dust_extinction` returns cm^2/H; convert with a factor 10^8 before combining them, as Eq. (4) requires.

Inlining. The loaded grids and integrals are also exposed `public, protected` (`scm_lambda`, `scm_theta_i`, `scm_theta_s`, `scm_phi`, `scm_theta_ran`, `scm_cext_tot`, `scm_csca_tot`, `scm_cext_ref`, `scm_csca_ref`, `scm_F_tot`, `scm_F_ref`, `scm_Z`, and the `scm_n*` sizes), so a host that prefers to inline its own lookups can take them once at startup and never call back. A complete two-cell reference consumer of every call is `sed/rt_example/use_dustlib_scatmat.f90` (`make use_dustlib_scatmat.x`); it demonstrates the band selection, Eq. (4), the aligned and random queries, and the closure checks.

6.6 Division of labor

Table 4 is the part to get right. SEDust stops at the intrinsic quantity; everything that depends on where the field points is left to the transfer code, because SEDust knows nothing about the cell geometry.

Done by SEDust	Left to the calling code
Integral over the grain size distribution	$\sin^2\gamma$, with γ the angle between the *local magnetic field* and the *line of sight* (not a sky-plane angle)
Weighting by the alignment efficiency $f_{\rm align}(a)$	Turbulent depolarization $F_{\rm turb}$
Summation over components (only astrodust is aligned; PAHs contribute zero)	Splitting into Q and U using the position angle ϕ of the projected field

Table 4: What is already integrated into `lamI_pol` and into `Cpol_ext` (equivalently, column 8), and what a radiative transfer host must still apply.

The observed Stokes emissivities follow from the exported quantity as

$$(j_Q,\, j_U) \;=\; \texttt{lamI\_pol}\,\sin^2\gamma\, F_{\rm turb}\,(\cos 2\phi,\, \sin 2\phi), \tag{5}$$

with j_I = `lamI_total` unchanged. The same $\sin^2\gamma$ and $F_{\rm turb}$ multiply `Cpol_ext` (column 8) when the extinction matrix is assembled. Equation (5) is stated here as a usage rule; its derivation is in §5.1 of `aligned_grain_polarization.pdf`.

6.7 Minimal example

```
use dust_lib
use radfield, only: J_Mathis
type(dust_model_t) :: m
real(wp), allocatable :: J_lam(:), lamI_total(:), lamI_pol(:), p(:)
integer :: i

call build_dust(m, 'astrodust', '../data')
allocate(J_lam(m%NLAM), lamI_total(m%NLAM), lamI_pol(m%NLAM), p(m%NLAM))

call J_Mathis(1.0_wp, m%lam, J_lam)
call dust_emission(m, J_lam, lamI_total, lamI_pol=lamI_pol)

! intrinsic polarization fraction (before any geometric factor)
do i = 1, m%NLAM
   if (lamI_total(i) > 0.0_wp) then
      p(i) = lamI_pol(i) / lamI_total(i)
   else
```

```
    p(i) = 0.0_wp
  end if
end do
```

Multiply p by $\sin^2\gamma\, F_{\rm turb}$ to obtain the polarization fraction of a real line of sight.

6.8 Limits

- **Astrodust only.** `build_dl07` and `build_zubko` have no polarized optics. `build_dl07` releases any polarized arrays left by a previous astrodust build, so `lamI_pol` returns zeros rather than stale values.
- **PAHs contribute zero** ($f_{\rm align} = 0$, an HD23 assumption). The polarized emission therefore vanishes in the mid infrared, where stochastically heated PAHs dominate.
- **Circular polarization: available only from the regenerated table.** The released optics table carries $Q_{\rm ext}$, $Q_{\rm abs}$ and $Q_{\rm sca}$ but no phase retardation, so with it $C_{\rm circ} = 0$. The regenerated orientation-resolved table (§6.4) adds a birefringence block, and `dust_extinction` returns it as the optional `Cbir_ext` (§5.); the aligned scattering table carries the same birefringence in its $K(\theta_i)$ (§6.5). Consuming it in the $U \leftrightarrow V$ transfer term is the host's task.
- **Scattering by aligned grains: now available.** The angle-resolved Mueller matrix of the aligned astrodust spheroid, which no released file contains, is computed by `run_scatmat_aligned.x` and served through the API of §6.5. The release-derived cross sections still supply only the integrated $Q_{\rm sca}$; the angular optics come from the T-matrix engine.
- **Far-infrared and submillimeter polarized emission is complete without a scattering matrix.** The albedo there is essentially zero, so the transfer needs only the extinction matrix and the emission vector, both fully determined by what is exported here.

Reference numbers. For a Mathis ISRF the intrinsic polarization fraction `lamI_pol/lamI_total` is 0.00% at 12 μm, 13.67% at 100 μm, 17.15% at 154 μm, 18.79% at 350 μm and 19.15% at 850 μm. Column 8 reproduces the released `polarized_extinction.dat` to a median of 0.0296%.

7. The extreme-ultraviolet extension

The astrodust model's wavelength grid is the grid of the T-matrix Q table it was built from, and *two* scalar tables ship side by side:

file	wavelengths	range [μm]
`q_astrodust_P0.20_Fe0.00_1.400.dat`	1129	0.0912 – 39810
`q_astrodust_P0.20_Fe0.00_1.400_euv.dat`	1762	10^{-4} – 39810

The second spans 12.4 keV down to 0.031 eV; the first is that file with every wavelength shortward of the Lyman limit dropped (`make lyman_cut` in `tmatrix/`), row selection only, so over the 1129 they share they are the same numbers

cell for cell. **Which one the host passes as `qtab` decides what its model covers.** Given the _euv table, the ionizing band is inside the table, computed the same way as the rest of it, so a host that transports ionizing radiation reads it off the table and needs nothing more — in particular it does not have to link `libtmatrix.a`. Given the default table, a model that needs the band builds one, and §7.3 is where its optics come from. The *orientation-resolved* table did not follow the scalar one into that band, and §7.4 is where that is stated.

Below 0.0912 μm the table's wavelengths are the DH21 dielectric function's own energy nodes rather than a resampling of them. That band is full of absorption edges, each tabulated as a step across a close pair of energies — the separations run from 2.4×10^{-6} to 3.6×10^{-4} in relative energy, and k jumps by +211% at Fe K (7124 eV), +131% at O K (538), +67% at Fe L (724), +30% at C K (291), +27% at Si K (1846) and +26% at Mg K (1311). Counting them takes a threshold, and the answer moves with it: 23 places where k jumps by more than 0.5% across a pair closer than 5×10^{-4}, of which 14 are closer than 10^{-4}. A uniform 200-per-decade axis would average every one of them into a ramp. Reusing the nodes keeps every edge an exact step between adjacent grid points, and makes the refractive index at each point the tabulated one rather than an interpolate.

One point is deliberately not a dielectric node: $0.0912(1-10^{-4})$, just below the Lyman limit. It is placed for the *radiation field*, not for the grain — see §7.1.

Both `build_astrodust` and `build_dl07` take an optional `lam_min` [μm] for a host that needs to reach below the table it was given:

```
call build_dl07(m, qtab, sizedist, NT, T_lo, T_hi, lam_min=6.21e-5_wp)
```

Log-spaced points are then prepended from `lam_min` up to the table, at a step no coarser than the table's own $\Delta \ln \lambda$ at its short-wavelength end (0.00794 on the _euv axis, 0.01156 on the other), with `m%lam(1)` set to `lam_min` exactly. The request is refused (`status = 4`) when it asks for a wavelength the model's own dielectric data does not cover, rather than served with a refractive index frozen at the table boundary. **Omitting `lam_min`, or passing one the table already covers, prepends nothing.**

What `lam_min` does for astrodust therefore depends on which table the model was built from. On the default 1129-wavelength table a `lam_min` below 0.0912 μm builds a band. On the _euv table there is nothing left to ask for: it already reaches the short-wavelength end of the DH21 dielectric function, so every `lam_min` either falls inside the table (no band) or below the dielectric data (refused). DL07 can be extended below either, to 6.205×10^{-5} μm, because the D03 optical constants reach further than both.

7.1 A step in the field needs a grid point, exactly as a step in the material does

An interstellar radiation field is exactly zero below the Lyman limit and finite above it: `J_Mathis` in `sed/src/radfield.f90` opens with `if (lambda(i) < 0.0912d0) J(i) = 0`, and immediately above the limit $J_\lambda = 3069\,\lambda^{3.4172}$ is 0.856. That is a step discontinuity of the *field*.

The DH21 dielectric energy axis has no node between 13.595 and 14.000 eV. Built from those nodes alone, the grid would jump from 0.088557 μm straight to 0.0912 μm — one cell 2.98% wide, sitting across the step. Every wavelength integral in the library is a trapezoid sum, so that cell would integrate a ramp rising from zero and count absorption in a band where the field carries no photon at all. Measured on the shipped optics, with $C_{\rm abs}$ from the Q table itself, it inflates

the absorbed power of the smallest grains by 1.74% (falling to 0.05% for $a \geq 0.3\,\mu$m, which take little of their heating in the ultraviolet). Small grains are the ones the stochastic solver treats, so the error does not stay small: it moved the equilibrium/stochastic boundary by twenty radii and shifted the emergent SED by up to 13% at the far-infrared peak. The extra grid point at $0.0912\,(1-10^{-4}) = 9.119088 \times 10^{-2}\,\mu$m closes that cell, and with it the artifact: $1.74\% \to 0.006\%$. Its refractive index is interpolated rather than tabulated, which costs nothing — the astrodust dielectric function is smooth across 13.6 eV and has no edge there. The Lyman limit is a property of the illumination, and this is the same move the extension already makes at every one of the material's own edges: resolve a step with a close pair of points.

7.2 Where the optics come from in the extended band

- **Astrodust grains.** The T-matrix on the Draine & Hensley (2021) dielectric function — the same material *and* the same $b/a = 1.400$ oblate spheroid as the Q table, on the same random-orientation average, so the band continues the table's own calculation instead of substituting a different particle for it. This is the default; passing `euv_tmatrix=.false.` instead selects Bohren–Huffman Mie for the *volume-equivalent sphere*, much cheaper but an approximation. Both are described in §7.3. On the _euv table **neither is ever reached**, because no astrodust band forms below it and the flag has no effect; on the default 1129-wavelength table a `lam_min` below the Lyman limit reaches them.

- **PAH / carbonaceous.** Above 21.4 eV ($1/\lambda = 17.25\ \mu\text{m}^{-1}$) the DL07 PAH cross section is zero by construction, so graphite is the *whole* carbonaceous absorption there. Below that energy the D16 turbostratic-graphite sphere table is used as before; above it the code switches to Mie on the D03 graphite dielectric functions (random orientation, $\frac{1}{3}$ parallel $+\frac{2}{3}$ perpendicular), because the D16 table's radius axis bottoms out at $10^{-3}\,\mu$m while the astrodust size grid starts at $3.55 \times 10^{-4}\,\mu$m. In the Rayleigh limit $Q_{\rm abs} \propto a$, so clamping a smaller grain to the table's first radius overestimates its absorption by $10^{-3}\,\mu\text{m}/a$ — up to $2.4\times$ for the smallest PAH the size distribution populates ($4.22 \times 10^{-4}\,\mu$m). *This branch is unreachable on the unextended grid*: its shortest wavelength gives $1/\lambda = 10.96 < 17.25$, so every previous result is unaffected.

- **DL07 model.** Its optics are Mie on the D03 dielectric functions at *every* wavelength, so nothing changes character across the band and the extension is a grid extension only.

- **Zubko model.** No extension, and none possible: its tabulated optics already span 10^{-3} to $10^{4}\,\mu$m, and those tables *are* the model definition, so there is no dielectric function to extend them with.

7.3 The two routes, and what the cheap one costs

Read this section as a description of a route that no longer has anything to compute for astrodust. It was written when the Q table stopped at the Lyman limit and every ionizing host needed a band solved on the fly. The table now covers that band, so the astrodust EUV path is unreachable — which is the point: no host has to link the T-matrix any more. The route is kept because it is what would compute a band if a future dielectric function reached further than the table built from it, and because the timings below are the measurement that justified moving the work into the table. In particular **the volume-equivalent-sphere substitution described here is used nowhere in the shipped astrodust**

products: every wavelength of every one of them is read off the Q table, and no argument to `build_astrodust` can change that.

The Q table is a random-orientation T-matrix solution for an oblate spheroid of axial ratio $b/a = 1.4$. **By default the extended band is the same solution**, called directly (`spheroid_q` in `tmatrix/driver/spheroid_optics.f90`) on the same dielectric function, with the same convergence tolerance and the same regime bounds the table's own driver uses: the Rayleigh dipole limit below $x = 2\pi a_{\rm eff}/\lambda = 0.1$, the T-matrix between 0.1 and 50, the geometric-optics limit above 50. Nothing about the particle changes across the seam.

That solution lives in `sed/src/euv_astrodust_tmatrix.f90`, the one file of the library that reaches into `libtmatrix.a`, and it is reached through a registered procedure rather than a compiled dependency — which is what lets the rest of the library link without the T-matrix at all (§2.). A build that does not carry it answers `euv_tmatrix = .true.` with `status = 11`; it never falls back to the sphere below, because that is a different particle and the caller has to know.

That is not free. Measured on this machine (`gfortran` 13.1, `-O3`, 167 radii; the timings are the band alone, as `sed_init` reports them):

`lam_min`	$n_{\rm euv}$	band points	of which T-matrix	threads	time
0.0247968 μm (50 eV)	113	18 871	12 204	1	527.5 s
”				2	264.2 s
”				4	136.7 s
”				8	69.2 s
1.001×10^{-4} μm (12.4 keV)	590	98 530	41 900	72	73.5 s

The second row is the floor `calc_kext.x astrodust euv` picks by default. Most of a deeper band is cheap, because past $x = 50$ it is the geometric-optics limit: going from 50 eV down to 12.4 keV multiplies the band points by 5.2 but the T-matrix points by only 3.4.

The band is built with OpenMP over grain radii, with one T-matrix workspace for each thread (37 MiB at allocation, 80 MiB once its full-direct arrays are in place; nothing is allocated when there is no band to compute). That is the one thing to watch on a many-core host: peak resident memory measured 64 MB / 110 MB / 197 MB / 376 MB at 1 / 2 / 4 / 8 threads and 3.0 GB at 72, so cap `OMP_NUM_THREADS` if memory is tight. Each radius writes its own column and reads no other, so the result does not depend on the thread count or the schedule — the four runs above wrote byte-identical optics.

The cheap route. `euv_tmatrix=.false.` replaces the spheroid by its volume-equivalent *sphere* and solves it with Bohren–Huffman Mie, in a tenth of a second. The justification for that substitution is **not** the anomalous-diffraction argument $|m-1| \ll 1$: for this material $|m-1| = 0.77$ at the seam and still 0.56 at 20 eV. It rests instead on the two limits that actually apply:

- **Small grains are in the Rayleigh limit** ($x = 2\pi a/\lambda = 0.033$ at the seam for the smallest populated radius, and only 0.24 at 100 eV). There the ellipsoid polarizability is $\alpha_j = V(\epsilon-1)/\{4\pi[1+L_j(\epsilon-1)]\}$, so shape enters *only*

through $L_j(\epsilon-1)$. And $|\epsilon-1|$ falls from 1.86 at 13.5 eV to 1.10 at 20 eV, 0.28 at 40 eV and 0.061 at 100 eV: the depolarization factors drop out and sphere and spheroid converge on the same $C_{\rm abs}$.

- **Large grains are in the geometric-optics limit**, where $C_{\rm ext} \to 2\times$(orientation-averaged projected area). By Cauchy's theorem that average is $S/4$, and for $b/a = 1.4$ the spheroid's $S/4$ exceeds $\pi a_{\rm eff}^2$ of its volume-equivalent sphere by 2.12%. Mie is therefore low by $1-1/1.0212 = 2.08\%$ in that limit. **The error converges to a constant, not to zero**, however far into the EUV one goes.

Consequently the error does *not* fall monotonically with photon energy. Only $a \lesssim 2\times10^{-3}\,\mu$m improves monotonically; $a \gtrsim 0.01\,\mu$m gets *worse* between 13.6 and 30 eV before recovering, and the largest sizes settle at a nearly energy-independent −1.9 to −2.0%. Over the whole band and the whole size range the error stays within about 2%; `sed/src/q_astrodust.f90` tabulates it size by size.

Measured against the default. Building the same model both ways with `lam_min` = 0.0247968 μm gives, for $\Delta C_{\rm abs}$ = (Mie − T matrix)/T matrix [%]:

a [μm]	13.8 eV	17.7	24.8	30.1	40.2	50
4.73×10^{-4}	−0.31	+0.71	−0.01	−0.08	−0.13	−0.02
1.00×10^{-3}	−0.14	+0.75	+0.05	−0.00	−0.03	−0.02
5.01×10^{-3}	−0.16	+0.75	−0.09	−0.17	−0.24	−0.23
1.00×10^{-2}	+0.30	+0.24	−0.60	−0.57	−0.37	−0.27
5.01×10^{-2}	−1.28	−1.47	−1.59	−1.58	−1.47	−1.27
1.00×10^{-1}	−1.62	−1.74	−1.85	−1.87	−1.86	−1.77
2.99×10^{-1}	−1.90	−1.96	−2.01	−2.02	(geometric optics)	

Over the $0.1 \le x \le 50$ block where the T-matrix itself answers, $\Delta C_{\rm abs}$ runs from −2.03 to +0.77% with a median of −0.70%, exactly the bound argued above. Where $x > 50$ both routes leave the T-matrix — the spheroid one for its geometric-optics limit, which is what the Q table uses there too — and the two disagree by −6 to −19%; that is Mie against a chord approximation, not against the T-matrix.

Seam continuity. Extrapolating the Q table's first two wavelengths log–log back to the last extended-band wavelength and comparing with what the band actually produced there, over all 167 radii:

route	median |step|	max |step|	RMS step
T-matrix (default)	0.012%	0.068%	0.027%
Mie sphere (`euv_tmatrix=.false.`)	1.167%	15.709%	6.186%

The default is continuous to a few hundredths of a percent, about a hundred times smoother than the sphere substitution. The asymmetry parameter behaves the same way: the table gives $\langle\cos\theta\rangle = 0$ wherever its own regime bounds put a radius in the Rayleigh or geometric-optics limit, and the default reproduces that exactly, whereas the sphere route steps from 0.90 to 0 across the seam for $a \gtrsim 1\,\mu$m.

The shipped extinction table. `data/astrodust/kext_astrodust_MW_euv.dat` now takes every wavelength from the Q table, the band included, so the extinction it serves and the emission the model computes are one calculation of one particle throughout — the property the paragraph below was written to establish, reached by extending the table rather than by solving a band. The comparison it records was made when the band was still computed on the fly: against the sphere route, the 1129 rows at $\lambda \geq 0.0912\,\mu$m were byte-identical, because those came from the Q table either way. Below the seam the sphere route was low in $C_{\rm abs}$ and $K_{\rm abs}$ by up to 3.6%, high in $C_{\rm sca}$ by up to 4.6% and in albedo by up to 5.1%, and wrong in $\langle\cos\theta\rangle$ by up to a factor of four at the shortest wavelengths, where the sphere keeps a forward-scattering asymmetry the spheroid's geometric-optics limit does not. $C_{\rm ext}$ moved least, by 0.72%. Regenerating costs about five minutes on 64 threads (`OMP_NUM_THREADS=64 ./calc_kext.x astrodust euv`). **The sphere route can no longer be selected here**: the table covers the band, so `build_astrodust` forms no band for `euv_tmatrix` to steer, and the comparison above cannot be reproduced without shortening the table first.

7.4 The polarized optics did not extend with the grid

This is the one place where the polarized branch has a hole, and it is announced rather than hidden. The scalar Q table was carried into the ionizing band; the orientation-resolved table it is paired with was not. That was a decision rather than an oversight: polarized transfer is run at a handful of wavelengths, not on a whole axis, so sweeping the 633 new wavelengths for all 169 radii would cost hours of solver time for entries no planned calculation reads.

A model built on the default 1129-point Q table has no such hole: its grid *is* the polarized table's. On the `_euv` 1762-point grid the polarized block covers the last 1129 wavelengths, 0.0912–39810 μm, and the 633 below 0.0912 μm have no polarized entry at all. `build_Cpol` locates the covered block from the polarized table's own coverage, looks for the companion table `q_astrodust_jori_euv_P0.20_Fe0.00_1.400.dat.gz`, and — since **that table does not ship** — says so on `error_unit`, on every build, whether or not `lam_min` was ever mentioned:

```
build_Cpol: no EUV polarized table (...)
          dichroic extinction and birefringence are zero below 9.120E-02 um
          (zero by omission, not by physics).
```

`Cpol_ext` and `Cbir_ext` are left at zero over that block. **A host must not read that zero as a physical result.** Measured from first principles (`tmatrix/driver/euv_polarized_optics.f90`), the alignment-weighted, size-integrated $|C_{\rm pol,ext}|/C_{\rm ext}$ is 1.26×10^{-3} at the 0.0912 μm seam, *rises* to 3.64×10^{-3} near 20.6 eV — a factor 2.9 — and decays to 1.6×10^{-4} at 100 eV, **with the sign opposite to the optical band**; the reversal falls near 0.106 μm. The scalar table could not have supplied these entries even in principle: it is a random-orientation average, and an orientation average carries no dichroism by construction.

To fill the band, generate the companion table with `tmatrix/run_q_jori.x` (§8.6) and pass the resulting pair as `qpol_euv_path` and `qpol_euv_wave_path`. With a table present, `build_Cpol` interpolates it in $\log\lambda$ and $\log a$, and a grid running outside its 0.0124–0.0912 μm coverage is refused (`status = 9`) rather than answered with an invalid limit. That table stops at 0.0124 μm (100 eV) because the opaque geometric-optics limit it leans on needs the chord optical depth $4\,{\rm Im}(m)x$ to be large, and for astrodust that is 3.7 at $x = 50$ there and falls below 1 by 200 eV.

The *scalar* optics are unaffected by any of this: $C_{\rm ext}$, $C_{\rm abs}$, $C_{\rm sca}$, `gbar` and `albedo` are complete over every wavelength

of whichever Q table the model was built from.

7.5 The single-photon ceiling comes from the field

The stochastic solvers set the top of a grain's enthalpy window from the energy of the hardest single photon it can absorb. That ceiling used to be the hard-coded Lyman limit, 13.6 eV, which an ionizing band makes wrong: a photon harder than the ceiling drives an excursion the window cannot hold. It is now

$$\max\big(13.6\,\mathrm{eV},\ hc/\lambda_{\mathrm{hard}}\big), \qquad \lambda_{\mathrm{hard}} = \min\left\{\lambda_i\ :\ J_\lambda(\lambda_i) > 10^{-12}\max_j J_\lambda(\lambda_j)\right\},$$

computed by `hardest_photon_energy` in `sed/src/radfield.f90` and called from all four places that need it — `sed_grain_loop` and `narrow_iterative` in `sed_astrodust.f90`, `qm_solve_grain` in `stoch_qm.f90`, and `calc_P` in `p_sub.f90`. Any units may be passed for J_λ, since only ratios within the array are used.
`calc_P` is the fourth because it does not merely bound a window: it integrates the upward transition rates over the photons that drive them, so both the cutoff and the limits of that integral are properties of the field. §7.6 is what it cost to have taken them from the grid instead.

The ceiling is a property of the field, not of the grid. That distinction is the whole point, and a coincidence hides it. A model's wavelength grid is the grid of its optics tables, which can reach far past the band a transported field occupies:

model	grid floor	implied photon energy
astrodust	$1.000\times10^{-4}\,\mu$m (T-matrix Q table)	12398 eV
DL07	$1.000\times10^{-4}\,\mu$m (the same table)	12398 eV
Zubko	$1.000\times10^{-3}\,\mu$m (ZDA optics tables)	1239.8 eV

Every shipped model now depends on the distinction: the grid floors above sit 912, 912 and 91 times harder than a Mathis field's own, which is the Lyman limit, and the field is the one that is right — it carries no photon at all below $0.0912\,\mu$m, however far the optics table extends. Before the Q table was carried into the ionizing band, astrodust and DL07 could not tell the two apart: their grid floor *was* the Lyman limit (13.595 eV < 13.6 eV, so the maximum returned the constant whichever was measured), and Zubko was the only model that exercised the difference.

Taking it from the grid costs resolution, not physics. `umax` sets the upper edge of a *fixed* number of enthalpy bins, so a ceiling 91 times too high starts the solve with bins 91 times too coarse. The refinement loops do contract the window once the top bin is unpopulated, but the contraction is guarded (`umax > 1.02*umaxlo` and `umax > 1.01*ub(jcut)`) and capped at `MAX_ITER = 10`, so it does not return to where it began; expansion carries no such guard. Measured in a host code on the Zubko model, with no `lam_min` requested at all: dust temperature +0.0705 K median and positive in every cell, the ten brightest SED bins −0.9 to −1.9%, the emission image 1.9% median and 5.4% maximum, total emitted energy −0.08%. A systematic redistribution at nearly conserved energy is what a coarser bin grid produces, and `docs/EUV_EXTENSION_HOST_REGRESSION.md` records the measurement.

7.6 The photon that was not there

The ceiling above bounds a *window*. `calc_P` does something stronger: it integrates the upward transition rates over the photons that drive them. Until this release it took the limits of that integral from the grid, and the result was not a loss of resolution but a source of energy that does not exist.

The highest-bin term is the rate into the top enthalpy bin, driven by every photon more energetic than that bin's gap ΔH. Two things went wrong.

- **The integral started at `lambda(1)`**, the short end of the optics table, rather than at the shortest wavelength the field occupies. On the grid ending at the Lyman limit the two coincided; carried to $10^{-4}\,\mu$m they differ by 912, and a flat 51-point rule left between 1 and 8 samples inside the illuminated band.
- **The flux at the gap wavelength was read through a clamping interpolator.** `interp1` returns $y(1)$ for any abscissa below its range. The top bin is placed above the single-photon bound by construction, so $hc/\Delta H$ was always shorter than the grid, and the clamp answered every time with J_λ at the grid edge instead of zero — on a grid ending at the Lyman limit, the full Mathis intensity there. *Every top-bin transition was driven by a photon flux the field does not carry.* When ΔH exceeded the grid's whole range the same clamp made the loop run backwards, subtracting a rate rather than adding none.

Both limits now come from `hardest_photon_energy`, and the band is skipped when ΔH exceeds the hardest photon. The sample count follows the band's width (`NWAV_MIN`, `NWAV_MAX`, `DLNWAV_TARGET` in `p_sub.f90`) instead of being fixed, so the step is bounded whatever the band. Re-running the old grid with only the ghost removed and then only the sampling changed separates the two: the sampling moves nothing at all (0.0000%), which says the flat 51 points had already converged once the limits were right; the ghost carries the entire change.

All three shipped models moved, and Zubko's move has nothing to do with the astrodust table — its grid has always begun at $10^{-3}\,\mu$m, so it carried the same defect independently. Against the previous products: astrodust up to 12.1% locally and −1.1 to +0.9% in integrated power, DL07 9.7% and −2.8%, Zubko 3.0% (6.0% with `euv`). Energy conservation, measured as emitted over absorbed power per H, improved from +0.236% to −0.057% for the astrodust S1 stage and held at −0.11% for S2.

Every SED this package produced before this fix contains photons that were not in the field, in the top enthalpy bin of every stochastically heated grain. Comparisons against those products have to account for it.

Why 10^{-12} of the peak, and not simply $J_\lambda > 0$. Exact zeros must be excluded — `J_Mathis` returns exactly zero below $0.0912\,\mu$m — but an exact positivity test is too fragile: a host that hands over denormal or roundoff-level residue in its unilluminated bins would push the ceiling back up by orders of magnitude. A component a fraction 10^{-12} below the peak of J_λ contributes at most that fraction of the photon absorption rate; allowing two decades for the run of $C_{\rm abs}$ across the illuminated band, the enthalpy states it could populate carry probability $\lesssim 10^{-10}$ of the peak, at or below the 10^{-13} tail level (`PMIN_UP`, `PMIN_UP_QM`) at which both solvers already truncate the window. Such a component cannot change a resolved excursion, and 10^{-12} still sits eighteen decades above the residue it is meant to reject.

The error is one-sided on purpose. Both refinement loops *expand* the window while the top bin is still populated, so an underestimated ceiling is corrected as the loop runs; the guarded contraction can stop short of where it began, so an overestimate is not corrected at all. `MAX_ITER = 10` bounds the correction in either direction. Overestimating is therefore the dangerous direction, which is why the threshold is set high enough to reject junk rather than as low as floating point allows.

`calc_sed.x zubko` exists to keep this path covered. The defect escaped `calc_sed.x astrodust` and `calc_sed.x dl07` because for those two models the grid floor and the field's short end are the same number: no driver in the tree ran the emission solvers on a model whose optics grid reaches past the illuminated band. `calc_sed.x zubko` (§8.3) closes that gap and is built by the default `make`. It prints the two candidates side by side, so the difference shows in the first lines of output:

```
$ ./calc_sed.x zubko # grid cut at the Lyman limit, Mathis field
 optics grid short end : 8.9984E-02 um -> 1.37784E+01 eV
 hardest photon in the field: 1.35946E+01 eV
 single-photon bound in use : 1.36000E+01 eV
$ ./calc_sed.x zubko euv # whole ZDA grid, still the Mathis field
 optics grid short end : 1.0000E-03 um -> 1.23984E+03 eV
 hardest photon in the field: 1.35946E+01 eV
 single-photon bound in use : 1.36000E+01 eV
$ ./calc_sed.x zubko euv hardfield # ... and a hard component added to the FIELD
 optics grid short end : 1.0000E-03 um -> 1.23984E+03 eV
 hardest photon in the field: 3.70141E+02 eV
 single-photon bound in use : 3.70141E+02 eV
```

The middle run is the one that matters: the optics grid reaches 1.24 keV and the ceiling must still stay at 13.6 eV, because the field stops there. Only `hardfield`, which puts photons in that band, may move it — and it does, to the same 370 eV the field itself carries. `hardfield` implies `euv` for that reason: on the cut grid there is no wavelength to put those photons on.

A model built without `lam_min` is unaffected. For astrodust and DL07 the maximum selects the 13.6 eV constant on the unextended grid either way, so those results are unchanged to the last bit.

7.7 What else the extension does, and does not, change

- **The Planck integrals are anchored to the table range.** The internal log-λ integration grid for $\kappa_B(T,a)$ keeps its step and its sample points over the optics-table range and merely *prepends* points below it, so widening the model grid cannot coarsen the sampling of the range that carries the integrand. Measured: with a $1.001\times10^{-4}\,\mu$m floor, κ_B is unchanged to round-off over the thermal range ($\max|\Delta\kappa_B|/\kappa_B = 3.4\times10^{-16}$ for $T \le 3000$ K, and exactly zero below ~ 2400 K); $\kappa_{\rm CMB}$ is bit-identical. The only nonzero difference is 1.1×10^{-8} at the very top of a 5000 K grid, where the Wien tail genuinely reaches into the extended band — that is real extra emission, not a sampling

artifact. Without the anchoring, spending a fixed budget of points on the widened interval would have moved κ_B by $\sim 10^{-4}$ for a purely numerical reason.

- **Stochastic heating by hard photons is not solved by this change.** The extension fixes the photon-energy mapping and the absorbed-energy accounting. It does not address the finite enthalpy grid: upward transitions above the top of that grid are accumulated into the highest temperature bin, and for the smallest PAHs a 54 eV photon already exceeds $H(5000\,\mathrm{K})$. Nor does it add photoionization, electron energy loss, fragmentation or PAH destruction. Treat the extended band as extinction and absorbed-energy physics, not as a validated hard-EUV PAH *emission* model.

7.8 How far each model can be checked

author-provided data	coverage	what it contains
Draine, `kext_albedo_WD_MW_3.1_60_D03` (DL07 / WD01)	10^{-4}–$10^{4}\,\mu$m = 1.24×10^{-4} eV–12.4 keV	λ, albedo, $\langle\cos\theta\rangle$, $C_{\rm ext}$/H, $K_{\rm abs}$, $\langle\cos^2\theta\rangle$
HD23 astrodust release (`extinction.dat`, `scattering.dat`, `wav.dat`)	0.1–$3\times10^{4}\,\mu$m = 4.1×10^{-5}–**12.4 eV**	absorption and scattering $\tau/N_{\rm H}$ only. **No** $\langle\cos\theta\rangle$ **product exists anywhere in the release.**
Zubko ZDA optics tables	10^{-3}–$10^{4}\,\mu$m = 1.24×10^{-4} eV–**1.24 keV**	$Q_{\rm abs}$, $Q_{\rm sca}$, $Q_{\rm ext}$, g for each radius

Table 5: What the model authors published, and how far it reaches.

The consequence is blunt: **the astrodust EUV band has no external reference at all**, because the HD23 release stops at 12.4 eV — exactly where the extension begins. What can be said is that the band is computed the same way the published part of the model is, from the same dielectric function and for the same particle (§7.3), and that it joins the table without a step. That is a consistency argument, not a validation.

The DL07 model *can* be checked, because Draine published the same model over the whole range. Comparing `data/dl07/kext_dl07_MW_euv.dat` with `kext_albedo_WD_MW_3.1_60_D03.all_2003` on the reference's own wavelengths (and counting only points where the reference grid is no finer than ours — elsewhere the comparison measures our grid, not our optics) gives, per decade in λ, mean $|\Delta C_{\rm ext}|/C_{\rm ext}$ of 0.056–0.86%, mean $|\Delta\,\mathrm{albedo}| \le 0.0019$ and mean $|\Delta\langle\cos\theta\rangle| \le 0.00054$. `calc_kext.x dl07 euv` prints that table. The two residual features are worth naming: the excluded points cluster at the X-ray absorption edges (Fe K, Si K, Mg K, Fe L, O K, C K), which the reference brackets far more finely than our $\Delta\ln\lambda = 0.0116$ log grid can resolve; and the largest single deviation, 7.9% in the 1–10 μm decade, is confined to the 6.2, 7.6 and 7.9 μm PAH C–C features, where Draine's 2003 table uses a different vintage of the PAH cross sections.

8. Standalone programs

Every program here is named for the quantity it computes — `calc_sed.x` for the emission SED, `calc_kext.x` for the size-integrated transport optics, `calc_qtable.x` for the stored cross-section tables, `calc_enthalpy.x` for the enthalpy tables — so that none of them can be mistaken for the build tool. `make` in `SEDust/sed/` builds the default set; the rest have make targets of their own. All of them are run *from* `SEDust/sed/`, because every data path is relative to `../`.
They also share ONE command-line system (§8.2): one vocabulary, one parser, one refusal and one filename-tag rule, with each program declaring which axes it has a referent for.

8.1 `calc_kext.x` — size-integrated transport optics

Builds a model and calls `size_integrated_extinction` on it, then writes the result under `../data/`. Every model goes through the same two calls, so the files it writes *are* the optics an RT host gets in memory — these are the files `dust_extinction` reads back (§5.).

```
./calc_kext.x astrodust [euv | <lam_min in um>]
./calc_kext.x dl07 [ld01] [euv | <lam_min in um>]
./calc_kext.x zubko [formula | table] [zda | mie_d03] [euv]
./calc_kext.x from_files <descriptor> [data_dir]
```

With no argument it prints its usage and stops. `euv` carries the grid down to whatever the model's own dielectric function covers; an explicit number requests a specific floor in μm. `zubko` defaults to `formula`. `from_files` takes the descriptor path, and the data directory defaults to the descriptor's own directory. `ld01` asks for the DL07 model with the Li & Draine (2001) carbonaceous absorption in place of the Draine & Li (2007) one — everything else the same, so a ratio of the two curves isolates that one cross section — and lands beside it as `../data/dl07/kext_ld01_MW[_euv].dat`, with the matching `/kext_ld01` group inside the model's own product. The 2003 reference table `kext_albedo_WD_MW_3.1_60_D03.all_2003` was computed with those cross sections, which is why the point-by-point comparison this program prints at the end is the sharper test for that run. For astrodust the `euv` floor now falls inside the Q table, so `./calc_kext.x astrodust` and `./calc_kext.x astrodust euv` cover the same grid and write the same transport curve; they differ in the field widths, the header, and the dichroic column the first one adds.
Columns are

```
lambda[um]  albedo  <cos>  C_ext/H  C_abs/H  C_sca/H
```

in μm and cm^2 per H nucleon, followed by $K_{\rm abs}$ [cm^2/g] and — for `kext_astrodust_MW.dat` only — an eighth column $C_{\rm polext}$/H (Table 3). The first four columns are in the order of Draine's `kext_albedo` tables, so a reader written for those files (for instance the `par%ion_dust_kext` reader of the MoCHII host, which takes four reals from each row and uses λ, albedo and $C_{\rm ext}$) parses these unchanged. Note the fifth column differs from Draine's: his is $K_{\rm abs}$ [cm^2/g], ours is $C_{\rm abs}$/H [cm^2/H], so no dust-mass normalization is folded into the transport columns.

The $K_{\rm abs}$ column. Every product carries it, for every model. It is $C_{\rm abs}/{\rm H}$ divided by the model's dust mass per H, Eq. (2), which the model reports through `dust_mass_per_H` — so the column and the library agree by construction rather than by a constant copied between them. The normalization is one wavelength-independent number, as well defined in the extreme ultraviolet as in the optical, and the header of each file states its value and where that model's grain densities came from. The one product that would not get the column is a file-defined model whose optics tables declare no density; then $M_{\rm dust}/N_{\rm H} = 0$, the program says so, and writes the six transport columns alone.

Which product carries the polarized column. Of the four products, only `kext_astrodust_MW.dat` carries the dichroic $C_{\rm polext}$/H. It is the *narrower* product, on the 1129 wavelengths its Q table has, and every one of them is a wavelength of `kext_astrodust_MW_euv.dat`, over which the two agree to the 5×10^{-8} its narrower fields can express, so what separates the two files is the eighth column and the field widths this one keeps as a regression reference.

Where the orientation-resolved table stops, the column does not stop — it goes to zero, and the header says so. The narrower product's 1129 wavelengths are the polarized table's own, so every row of it carries a measured $C_{\rm polext}$/H. The `_euv` product reaches 633 wavelengths further, and there the column is exactly zero: a deficit, not a measurement. The polarized extinction in that band is of order 10^{-3} of $C_{\rm ext}$ and reverses sign, so zero is not even the right order (§7.4). Its header states the row count and the wavelengths, taken at run time from the polarized table's own coverage, and `build_Cpol` says the same on `error_unit` at every build.

Only the astrodust products carry column 8 at all; the DL07 and Zubko products are seven columns, because those models have no polarized optics. That eighth column is also the one thing in which the scalar branch (v1.00) and this one differ on disk: v1.00 writes the same seven transport columns, value for value, and stops there.

After writing, the program checks $C_{\rm ext} = C_{\rm abs} + C_{\rm sca}$ and the ranges of albedo and $\langle \cos\theta \rangle$, and — for astrodust and DL07 — prints the comparison against the author-provided data of Table 5.

These files are transport optics, not a dust model. They carry no size-resolved $C_{\rm abs}(\lambda, a)$, no grain enthalpy and no Planck integral, so stochastic heating and thermal emission cannot be computed from them. A host that needs emission must build the model and take the same curves from `dust_extinction`, which is what keeps the absorbed power and the re-emitted spectrum referring to one and the same grain.

8.2 The command-line system

Every program in `sed/` takes ONE vocabulary of run settings (`sed/src/sed_run_options.f90`), so that a word names the same physics whichever program reads it. The settings fall into independent *axes* (Table 6), and each program DECLARES which axes it has a referent for before it reads a single argument. Three things follow from that declaration:

- a word of a declared axis is read;
- a word of an axis the program did *not* declare is refused, naming the axis — `./calc_kext.x dl07 qm` answers `'qm' selects the stochastic-solver axis, which calc_kext has no referent for` — rather than being silently dropped, which is the failure this system exists to prevent. When the axis exists in the program but

not in the model it was asked about, the model is named instead: `./calc_sed.x zubko c2` answers `. . . which calc_sed's zubko has no referent for`;

- anything else is the program's own positional argument (a DL07 submodel name, a wavelength floor, a descriptor path).

axis	words	programs that declare it
subject (first argument)	the model, or the product, the program works on	all but `calc_polext.x`
solver	`heuristic` (default), `draine`, `equil`, `qm`, `qm_dbcon`, `qm_stati`	`calc_sed.x`
grid	`euv`	`calc_sed.x`, `calc_kext.x`
field	`mathis_orig`, `logU=X`, `hardfield`	`calc_sed.x`
emission	`induced`, `photcut`	`calc_sed.x`
matrix size	`nstate=N`, `nisrf=N`	`calc_sed.x`
graphite of the ξ blend	`gra_d03_sphere`, `gra_d16_sphere`, `gra_d16_spheroid`	`calc_sed.x` (astrodust, dl07)
carbonaceous vintage	`dl07` (default), `ld01`	`calc_sed.x` and `calc_kext.x` (dl07)
Stage-1 enthalpy	`c2`	`calc_sed.x` (astrodust), `calc_enthalpy.x`
Zubko size distribution	`formula` (default), `table`	`calc_kext.x` (zubko)
Zubko optics set	`zda` (default), `mie_d03`	`calc_kext.x` (zubko)

Table 6: The run-setting axes, and which program has a referent for each. Any combination across axes is a valid run.

Two kinds of combination are refused rather than resolved silently. Naming two values of one axis — two solvers, or two graphite sources — would leave the run doing one thing and its filename claiming another. So would naming a setting the chosen solver does not read: `nstate=` and `nisrf=` size the energy-space transition matrix, which only `qm`, `qm_dbcon` and `qm_stati` build, and `photcut` acts inside the bin sum of the heuristic narrowing solver, the only solver that reads it.

Every setting tags the output filename, so a variant run cannot overwrite the production one. Parsing and tagging are separate steps: reading an argument only records what was asked for, and the tag is assembled afterwards in one fixed order — grid, field extent, solver, graphite, optics set, enthalpy, field normalization, emission term, numerical sizes — so `induced qm` and `qm induced` write the same file. A default run carries no tag at all, which is what leaves the production filenames as they were. Each program prints the file it will write before it starts, so a mistyped setting is caught then rather than after the work.

Where a setting lands is a separate module (`sed_apply_options.f90`), because the answer is not the same for every program: one that computes an enthalpy table or a cross-section table links no solver and no radiation field, and would otherwise have to drag both in to get the shared command line.

8.3 `calc_sed.x` — emission SEDs

One program for the three shipped models: `./calc_sed.x <model> [settings . . . ]`, the model name required and everything after it from the system of §8.2. It writes `output/sed_<model>[_<submodel>][_<tag>][_<stage>].dat`.

`astrodust` — the production SED. Solves the astrodust+PAH SED for a single Mathis-ISRF cell at $U = 1.585$ ($\log U = 0.20$, the HD23 best fit), on a 200-point log T grid from 2.7 to 5000 K. Reads the T-matrix Q table `../tmatrix/output/q_astrodust_P0.20_Fe0.00_1.400.dat` and `../data/release/size_distribution.dat`; writes `output/astrodust_irem_ours_<tag>S1.dat`, `. . . S2.dat` and `. . . PAH.dat`, each with λ [μm] and $\lambda I_\lambda / N_\mathrm{H}$. Optional arguments, in any order: the shared settings of §8.2, plus the two axes only this model has a referent for — the graphite of the PAH ξ blend and the Stage-1 enthalpy prefactor `c2`.
`gra_d03_sphere`, `gra_d16_sphere` and `gra_d16_spheroid` name the graphite optics the carbonaceous PAH-to-graphite ξ blend takes. `'d16_sphere'` is the production choice and the HD23 one — Draine 2016 turbostratic graphite, Maxwell-Garnett effective medium, on a sphere — and naming it outright changes nothing. `gra_d03_sphere` takes the Draine 2003 dielectric functions with Mie on a sphere, weighting $E \parallel c$ by $\frac{1}{3}$ and $E \perp c$ by $\frac{2}{3}$, which is the original DL07 choice; `gra_d16_spheroid` takes the same D16 material on Draine's $b/a = 1.4$ oblate spheroid table, random-orientation averaged. They tag their output `d03gra_`, `d16gra_` and `sphdgra_` (`output/astrodust_irem_ours_d03gra_S1.dat` and so on). All three reach the PAH population only, so the S1 and S2 files they write are byte-identical to the production ones; what they move is the PAH sub-mm.

`dl07` — the DL07 reference SED. Solves the Draine & Li (2007) silicate + carbonaceous SED with the WD01 size distribution, at $U = 1$ unless `logU=X` asks for another intensity. `./calc_sed.x dl07 [model] [settings . . . ]`, where `model` is one of `mw31_00 . . . mw31_60` (default `mw31_60`), `lmc2_00`, `lmc2_05`, `lmc2_10`, `smc`; an unknown name is rejected with the valid list and the settings this driver takes. Those are the shared settings of §8.2 plus the graphite of the ξ blend, which this model computes as well — its own choice being `'d03_sphere'`. Writes `output/dl07_sed_ours_<model>[_<tag>].dat` with columns λ, total, silicate and carbonaceous $\lambda I_\lambda / N_\mathrm{H}$.
Naming a graphite source has one further effect here. The stored cross sections under `../data/dl07/` were computed once, with this model's own `'d03_sphere'` graphite, so reading them back would leave the request with no effect at all; a named graphite therefore solves every optic from the dielectric functions instead. That is what a comparison of two graphite choices needs anyway — both variants then take the same route — it costs about a second on the shipped grid, and `gra_d03_sphere` reproduces the stored-table default to every printed digit.

`zubko` — the Zubko/ZDA SED, and the wide-grid regression. Solves the Zubko, Dwek & Arendt (2004) BARE-GR-S SED — PAH + graphite + silicate, the ZDA size-distribution formula, the ZDA optics and calorimetry tables — for a Mathis ISRF at $U = 1$, and writes `output/zubko_sed_ours[_<tag>].dat` with columns λ, total, PAH, graphite and silicate $\lambda I_\lambda / N_\mathrm{H}$.

```
./calc_sed.x zubko [settings ...]
```

The settings are the shared vocabulary of §8.2; there is no graphite axis here, because this model is its distributed optics tables and computes no ξ blend of its own. The solver defaults to `heuristic`, the model default. `euv` builds the model

on the whole ZDA optics range, $10^{-3}\,\mu$m (1.24 keV) upward, instead of cutting it at the Lyman limit where the Mathis field stops. `hardfield` then replaces the field below the Lyman limit — where the Mathis field is identically zero — by a diluted 10^5 K blackbody, so that the field really does carry photons above 13.6 eV; the dilution is chosen so that band deposits roughly as much power as the whole Mathis field — enough for the hard photons to matter, little enough that the small grains still heat stochastically. It implies `euv`, a cut grid having no wavelength to put those photons on.

This is the emission counterpart of `./calc_kext.x zubko`, which exercises only the extinction size integral. It exists because Zubko is the one shipped model whose optics grid ($10^{-3}\,\mu$m) reaches far past the band a transported field occupies ($0.0912\,\mu$m for the Mathis ISRF), so anything in the stochastic-heating path that reads the grid where it should read the field shows up here and nowhere else. Running it with and without `euv` is the regression of §7.5.

8.4 `calc_enthalpy.x` — enthalpy tables (diagnostic)

Tabulates $U(T, a_{\rm eff})$ for the two astrodust enthalpy stages and writes `output/enthalpy_S1.dat` and `output/enthalpy_S2.dat` (201 log-spaced temperatures from 2.7 to 5000 K $\times$ the 169 radii of `../data/astrodust/DH21_aeff`, T outer and a inner). Its one axis is `c2`, the Stage-1 density-corrected prefactor, which tags its table (`output/enthalpy_S1_c2.dat`) and writes no Stage-2 file, Stage 2 not reading that setting. **It is not required to compute an SED**: the solver calls the closed-form Debye sums directly in its inner loop. Use it to compare the two stages, to plot $U(T)$, or to spot-check against published DL01 values.

8.5 Polarized and diagnostic programs

Four more targets are built on request rather than by the default `make`: `calc_polext.x` (polarized extinction per H, checked against the HD23 release), and the three self-checking test programs `test_scalar_mode.x`, `test_scatmat_aligned.x` and `test_euv_extension.x`. The tests print `ALL CHECKS PASSED` or the failing check; build them explicitly, since `make` alone will not refresh a stale binary:

```
make test_scalar_mode.x test_scatmat_aligned.x test_euv_extension.x
```

8.6 Polarized optics at other wavelengths

These programs live in `SEDust/tmatrix/`, not `sed/`, and are run from there. They exist because **the library reads tables and never runs the T-matrix at run time** — not to keep the code small, but because convergence cannot be certified above $x \simeq 55$ (below), and a silently wrong value is worse than an absent one. A revision of the T-matrix core is planned, so read this as the current policy rather than a permanent one. The core has since been rewritten as a re-entrant modern-Fortran library, but that change was structural and its output byte-identical; the convergence ceiling it was about is untouched.

Polarized *transfer* at every wavelength is rarely wanted; the five optical bands that ship cover the usual case. What matters is that any other wavelength can be computed:

```
./run_q_jori.x lam L1 [L2 ...] [ja=JA1:JA2] [tag=NAME]
./run_q_jori.x lamfile PATH [ja=JA1:JA2] [tag=NAME] # one per line, '#' comments
```

```
./run_q_jori.x lammerge STEM FILE [FILE ...] # reassemble the ja= windows
```

Each writes a pair, `q_astrodust_jori_P0.20_Fe0.00_1.400.TAG.dat` and `.TAG.wave`, which is exactly what `qpol_euv_path` and `qpol_euv_wave_path` take. Because `build_Cpol` interpolates in $\log\lambda$, the pair needs **at least two wavelengths**.

`ja=JA1:JA2` splits the 169 radii across *processes*, not threads. That was once an engine constraint: the f77 solver handed the converged T-matrix to `AMPL` through `COMMON /TMAT/` and kept further working storage in COMMON blocks, two of them blank, so the core was not re-entrant. None of that state is left — every working array is in a caller-owned `tmatrix_workspace_t`, one active caller for each workspace, and distinct workspaces may be evaluated concurrently — but `run_q_jori.x` keeps the process windows, which are already measured. Measured on this machine: 9m22s of wall time for 34m48s of processor time, 57 processes over three wavelengths.

Convergence is certified, not assumed. In `lam`, `lamfile` and `euv` mode every node with $x \in (50, 60]$ is solved twice — (DDELT, NDGS) = $(10^{-3}, 2)$ and $(3\times10^{-3}, 3)$ — and rejected if the two disagree by more than 5%, in which case the geometric-optics limit is used instead. Above $x = 60$ that limit is used outright. This matters because **`IERR = 0` is not a sufficient condition for convergence above** $x \simeq 55$: at $\lambda = 0.0912\,\mu$m, $a = 0.9441\,\mu$m ($x = 65.04$) the solver returns `IERR = 0` together with a value about 50% away from its neighbors. At the seam $x = 51.7$ and 54.7 pass the cross-check while 57.9, 61.4, 65.0 and 68.9 fail. The older full-sweep and `range` modes keep the previous $x > 50$ rule, so they still reproduce the shipped table byte for byte.

What certification costs. Leaving $x > 60$ at the geometric-optics zero discards part of the dichroic extinction: nothing at the seam, $\sim 8\%$ of the band total at $0.031\,\mu$m, $\sim 20\%$ at $0.022\,\mu$m and $\sim 46\%$ at $0.0124\,\mu$m. The true value lies between the certified 1.06×10^{-4} and the $1/x$ extrapolation 3.14×10^{-4} — already about 1% of the 5.5×10^{-2} the V band reaches, so the residual is a small fraction of an already small number. $C_{\rm pol}$, which drives polarized *emission*, has no such gap: the geometric-optics limit obtains it from the opaque-grain Fresnel surface integral.

Scattering matrices are less flexible. `run_scatmat_aligned.x` already accepts wavelengths on the command line (`./run_scatmat_aligned.x 0.44 0.55 0.79`), but three things `run_q_jori.x` has are missing: its output stem is fixed, so a custom run **overwrites** the shipped five-band table; there is no merge mode, so bands cannot be *added* to an existing table (a full recomputation costs about 4.5 minutes per band); and it has neither a radius split nor the $x > 50$ cross-check above. Adding a band today therefore means regenerating the whole table and accepting the uncertified large-x treatment.

9. The model object and accessors

```
type :: dust_model_t
  character(len=32) :: name
  integer :: NA, NLAM, NT
  real(wp), allocatable :: lam(:) ! (NLAM) [um]
```

```
  real(wp), allocatable :: T_first(:) ! (NT) [K]
  type(grain_pop_t), allocatable :: pops(:)
  integer :: n_channel
  character(len=16), allocatable :: channel_name(:)
  logical :: use_induced_emission ! default .false.
  character(len=16) :: stoch_method ! default 'heuristic'
  logical :: verbose ! default .false.
end type
```

Diagnostics. `m%verbose` (default `.false.`) keeps the library solve path silent; set it `.true.` to print the same solver diagnostics as the CLI drivers.

Convenience accessors (all in `dust_lib`):

```
n = dust_nlam(m) ! number of wavelengths
nc = dust_n_channel(m) ! number of output channels
lam = dust_lambda(m) ! allocatable copy of the wavelength grid [um]
nm = dust_channel_name(m, ic) ! name of channel ic
md = dust_mass_per_H(m) ! dust mass per H nucleon [g/H]
```

`dust_mass_per_H` is the conversion from the library's cross sections per H nucleon to a mass opacity, Eq. (2) and §5.. Each population carries the solid density of its material in `m%pops(ip)%rho_bulk` [g cm^{-3}]; a population left at 0 contributes nothing to the sum.

10. The generic file loader

`build_from_files` reads a small text *descriptor* and assembles a model from data files, with no code changes. One `pop:` line per population:

```
# descriptor.txt
name = mymodel
# pop: <grain_type> <channel> <optics> <dnda_table> <calorimetry> <rho(0=use file)>
pop: pah 1 PAH_28_1201_neu.dat SzDist_PAH.dat Graphitic_Calorimetry.dat 0.0
pop: gra 2 Gra_121_1201.dat SzDist_Gra.dat Graphitic_Calorimetry.dat 0.0
pop: sil 3 suvSil_121_1201.dat SzDist_Sil.dat Silicate_Calorimetry.dat 0.0
```

- `grain_type`: `'sil'`, `'pah'`, or `'gra'` (selects the heat-capacity/mode model used by the `'qm'` solver; ignored by the GD solvers, which use the supplied enthalpy directly).
- `channel`: integer output channel (1 . . .); populations sharing a channel are summed.
- `optics`: a ZDA Q-table ($Q_{\rm abs}$ per radius and wavelength); $C_{\rm abs} = Q_{\rm abs}\pi a^2$.
- `dnda_table`: a 2-column table a [μm] and dn/da [cm^{-1} H^{-1}].

- `calorimetry`: a specific-enthalpy table $u(T)$ [erg/g]; $H(T,a) = u(T)\,\rho\,\frac{4\pi}{3}a^3$.
- `rho`: grain density [g cm^{-3}]; `0.0` uses the value in the optics-file header. The same number sets the population's `rho_bulk`, so it fixes both the enthalpy above and the model's dust mass per H, Eq. (2); an optics file that declares no density and a descriptor that supplies none leave the population out of that mass.

All files are sought under `data_dir`. The coded builders (`build_astrodust/dl07/zubko`) are the recommended path for the built-in models; `build_from_files` is for adding new data-defined models.

11. Linking into a 3D RT code

Compile your RT source against `libsedust.a` with the `.mod` search path pointing at `SEDust/sed/`:

```
gfortran -I<sed> my_rt.f90 <sed>/libsedust.a -fopenmp -o my_rt.x
```

Three complete examples are provided under `SEDust/sed/rt_example/`, each with its build command in its own header. No `ISO_C_BINDING` is needed — the RT code simply uses the Fortran module `dust_lib`.

- `use_dustlib.f90` — the scalar host flow, and the one to read first: build a model once (which is also where its extinction table is attached), take the transport optics from `dust_extinction` (including `gbar` and `albedo`), take one cell's emission from `dust_emission`. Step (4) then shows a host that transports ionizing radiation reading that band straight off the `_euv` Q table with no `lam_min` at all, and step (5) shows `lam_min` on DL07, the model whose dielectric data still reaches past the table (§7.). It is byte-for-byte the same file in the scalar branch (v1.00).
- `use_dustlib_pol.f90` — where SEDust stops and the RT starts: `lamI_pol` and `Cpol_ext`, and the $\sin^2\gamma$, $F_{\rm turb}$ and position-angle factors the host must supply (§6.6).
- `use_dustlib_scatmat.f90` — the aligned-scattering query API along a two-cell photon path (§6.5).

Threading. `m` is read-only during `dust_emission`; all scratch is local. The solver uses OpenMP internally over grains. If your RT parallelizes over cells, take care to avoid oversubscription (nested parallelism); the default `'heuristic'` solver is serial per call, which suits an outer cell-level parallel loop.

12. Units and conventions

- Wavelengths `lam` are in μm throughout; cross sections in cm^2.
- `J_lam` is a mean intensity with the same units as the Planck function B_λ (specific intensity per steradian). The library's output $\lambda I_\lambda / N_{\rm H}$ is then in erg s^{-1} cm^{-2} sr^{-1} H^{-1}.
- The CMB (2.725 K) is included in the heating field by `J_Mathis`.
- The induced-/stimulated-emission factor is off by default (the output is *net* emission, matching the HD23 and DL07spec released SEDs).

13. The HDF5 optics products

Every model's optics ship as one HDF5 file,

```
data/astrodust/sedust_astrodust.h5   data/dl07/sedust_dl07.h5   data/zubko/sedust_zubko.h5
```

holding that model's wavelength axis, its $(\lambda, a_{\rm eff})$ cross-section tables and its size-integrated extinction curve. One file per model, because the models share no grid, no radius grid and no component list, and because a host ships only the models it runs. This is the primary form; the text products (§14.) are still written beside it and stay readable with an eye and diffable.

13.1 One wavelength axis, and what `include_euv` means

Each file carries *one* wavelength axis — the widest the model has — and records `i_lyman`, the index of the *last* wavelength at or shortward of the Lyman limit, 0.0912 μm:

```
/ attrs: model, generator, layout
/grid/lambda (n_lam) [um], strictly increasing
/grid attrs: i_lyman, lambda_lyman_um, i_lyman_meaning
/kext/{albedo,g,C_ext,C_abs,C_sca,K_abs} (n_lam)
/kext attrs: M_dust_per_H, size_dist_file, text_product, method
/qtable/<comp>/a_eff (n_a) [um]
/qtable/<comp>/{Q_ext,Q_abs,Q_sca,g} (n_lam, n_a) Fortran = (n_a, n_lam) in h5py
/qtable/<comp>/regime astrodust only: 0 T-matrix, 10 Rayleigh, 20 geometric optics
/qtable/<comp> attrs: rho_bulk_g_cm3, method, source, scatters
```

A reader given `include_euv = .false.` — the default — returns `lambda(i_lyman:)` and the same slice of every wavelength-indexed array; `.true.` returns the whole axis. The narrow product is therefore a *view* of the wide one rather than a second file to keep in step with it: the two cannot disagree, and there is nothing to regenerate twice. That is what replaces the paired `*_euv.dat` text products. The cut is an HDF5 hyperslab, so a non-ionizing host reads only the rows it keeps.

Measured `i_lyman`: astrodust 634 of 1762, DL07 695 of 1823, Zubko 336 of 1201 — giving 1129, 1129 and 866 non-ionizing wavelengths, the lengths of the corresponding text products.

The rule is the *last* node at or below the limit, not the first at or above it, because the non-ionizing view is a table an RT host interpolates onto its own grid: it has to *cover* the band the host transports. The astrodust and DL07 axes carry 0.0912 μm exactly, so for them the two rules return the same index. The ZDA grid does not, and its first node above the limit sits 1.2×10^{-5} of itself inside it — just past `dust_extinction`'s edge allowance, so the same call that was served for two models used to be refused for the third. One extra node of the model's own table is not extrapolation; widening the allowance instead would have weakened the guard for every model at every wavelength.

model	components	n_a
astrodust	`astrodust`, `pah_neu`, `pah_ion`	169
dl07	`sil`, `gra`, `pah_neu`, `pah_ion`	84
zubko	`sil`, `gra`, `pah`	121, 121, 28
	`sil_mie_d03`, `gra_mie_d03`, `pah_mie_d03`	121, 121, 28

Table 7: Grain populations each product carries. Astrodust does *not* split into silicate and graphite: one composite grain plus PAHs is the premise of HD23, and the `'sil'` grain-type tag its populations carry internally is a legacy label for the enthalpy route, not a claim about the material. A population whose cross sections are an absorption prescription rather than a Mie solution carries `Q_abs` alone, and its `scatters` attribute says so, so that an absent scattering term cannot be mistaken for a computed zero. Zubko carries *two* sets on one axis — see §13.2.

13.2 Zubko carries two sets of optics

The ZDA BARE-GR-S model is defined by Zubko, Dwek & Arendt (2004), and the paper is where it would have to be read from. What this tree ships beside it are the three grain-input files the Camps et al. (2015) benchmark distributes —

```
data/zubko/suvSil_121_1201.dat data/zubko/Gra_121_1201.dat
data/zubko/PAH_28_1201_neu.dat
```

byte-identical to `SHG_Benchmark/DustModel/GrainInputs/` — so that the participating codes all run one agreed reproduction. Their headers record how they were made: Zubko's multilayer-sphere code (1997–2002), converted by K. Misselt in 2002, for the silicate and graphite; a 2009 implementation by Misselt of Li & Draine (2001) / Draine & Li (2007) for the PAHs.

Those files are the **default**. They are stored in `/qtable` of the product as they stand — `check_build_dust.x` reads both sides and requires an exact match — so a run against that benchmark measures the stochastic-heating solver and not an optics difference. This tree's own recomputation, Mie on the Draine (2003) optical constants, sits beside them under names suffixed `_mie_d03`, and each set has its own extinction curve so that what `dust_extinction` serves is the size integral of the very optics the model was built on:

```
/qtable/{sil,gra,pah} the benchmark's tables <- default
/kext ... and their size integral
/qtable/{sil,gra,pah}_mie_d03 Mie on the D03 constants
/kext_mie_d03 ... and their size integral

call build_dust(m, 'zubko', '../data', status=st) ! default
call build_dust(m, 'zubko', '../data', status=st, zubko_optics='mie_d03')
```

`build_zubko` takes the same choice as `optics`, and `calc_kext.x zubko [euv] [zda|mie_d03]` writes the matching curve. A name that is neither is refused (`build_zubko` status 8, `build_dust` status 92) rather than silently defaulted.

The recomputation reproduces the distributed silicate to a mean relative 2.4×10^{-6} and the graphite to 0.45% in $Q_{\rm sca}$ and

8% in $Q_{\rm abs}$. Its PAH component is a different implementation of the same LD01/DL07 prescription; measured against `Gra_121_1201.dat` cell by cell, the distributed PAH file holds graphite $Q_{\rm sca}$ and $\langle\cos\theta\rangle$ at every size and all 1201 wavelengths, and graphite $Q_{\rm abs}$ shortward of 0.0585 μm (21.2 eV, the DL07 PAH-to-graphite transition), so the ZDA PAHs scatter as the graphite sphere of their own radius. The recomputation now does the same. Through one builder and one size distribution the two sets differ by at most 0.17% in $C_{\rm sca}$ and 8.4% in $C_{\rm abs}$, the latter in the far-infrared where the graphite $Q_{\rm abs}$ discrepancy sits.

13.3 `build_dust`: one entry point

```
use dust_lib, only: dust_model_t, build_dust
call build_dust(m, 'zubko', '../data', include_euv = .false., status = st)
```

One call for `'astrodust'`, `'dl07'`, `'zubko'` and `'from_files'`, built from one directory and one flag:

```
call build_dust(m, model, data_dir [, NT, T_lo, T_hi] [, include_euv] [, status] &
              [, message] [, lam_min] [, kext_path] [, sd_index] [, u_isrf] &
              [, zubko_optics] &
              [, sizedist_path] [, config_path] [, astrodust_index_path] &
              [, euv_tmatrix])
```

`NT`, `T_lo` and `T_hi` are the internal temperature grid and are optional, defaulting to `200, 2.7, 5000` — the values the rest of this manual quotes as typical. They are what the EMISSION is solved on: the enthalpy $H(T,a)$ and the Planck-averaged opacity $\kappa_B(T,a)$ are tabulated on that grid, `calc_Teq` interpolates $T_{\rm eq}$ in it, and the stochastic solver's window is CLAMPED to $[T_{\rm lo}, T_{\rm hi}]$ rather than extrapolated past it — so the range must bracket the temperatures the field produces, and `NT` sets how finely $T_{\rm eq}$ is resolved. The extinction reads none of it, so a host that only wants `dust_extinction` can leave all three out.

`data_dir` is the SEDust data directory. The optics come from `data_dir/<model>/sedust_<model>.h5` — the wavelength axis, the cross-section tables and the extinction curve alike. What is *not* optics still comes from beside it: the size distribution (`data_dir/release/size_distribution.dat`, or the ZDA config) and the calorimetry.

`include_euv` and `lam_min` each mean *one* thing, for every model. `include_euv` selects the view: `.false.` cuts the model's own axis at `i_lyman`, so the grid a host gets covers the Lyman limit whatever model it named. `lam_min` states the shortest wavelength the model must *cover*, and never truncates: astrodust and DL07 meet it by extending on the dielectric function their optics come from, while Zubko and a file-defined model, which have no dielectric function behind their tables, meet it when those tables already reach and *refuse* it when they do not.

Both used to be model-dependent, and a host with one call and one physically motivated floor met the difference: `include_euv` was an index cut for two models and a wavelength cut for the third, and `lam_min` extended two models and truncated the other two.

The four builders of §3. are unchanged and still exported, so a host that names its own files keeps working. Two of them gained an optional argument that `build_dust` uses: `include_euv` on `build_astrodust` (which axis to take out of an HDF5 Q table) and `lam_axis` on `build_dl07` (the model's wavelength axis outright). The second is why a host

running DL07 alone needs no astrodust product: that model takes only a *grid* from a Q table, so `build_dust` hands it `/grid/lambda` out of `sedust_dl07.h5` and no Q table is opened.

13.4 The existing builders take an HDF5 path too

Every place that names an optics file dispatches on the `.h5` suffix, so no caller needs a second flag to say which form it meant:

```
call build_astrodust(m, '../data/astrodust/sedust_astrodust.h5', sizedist, NT, T_lo, T_hi, &
                  status=st, include_euv=.false.)
```

The same holds for `kext_path`. And with `kext_path` absent, a builder now tries `../data/<model>/sedust_<model>.h5` *first* and falls back to the text products listed under §14., and reads them the same way. `/kext` is read on the *whole* wavelength axis, never the non-ionizing slice: the curve is interpolated onto the model grid, so the widest one covers every grid the model can be built on. This is what retires the pairing of a narrow model with an `_euv`-only table, which used to fail at `dust_extinction` rather than at the build.

13.5 The polarized groups

The polarized branch adds three groups to the astrodust product, written by `calc_polarized_optics.x` (§13.6):

```
/polarized/lambda (n_lam_pol) -- its OWN axis
/polarized attrs: covers_euv, pol_valid_from, covers_euv_meaning
/polarized/qjori/{a_eff,Q_ext,Q_abs,Q_sca[,Q_re]} (3, n_a, n_lam_pol) in h5py
/polarized/qjori attrs: jori_convention, method, source, falign_*
/polarized/scatmat_random/{band_lambda,theta,F_tot,F_ref,C_sca_tot,C_sca_ref,
                        C_ext_tot,C_ext_ref}
/polarized/scatmat_aligned/{band_lambda,theta_i,theta_s,phi,Z,
                        C_ext_al,C_pol_al,C_bir_al,C_sca_al,C_sca_pol_al}
/polarized/scatmat_aligned attrs: profile_name, f_max, a_align, alpha
```

These carry their *own* wavelength axis and are never sliced by `/grid`'s `i_lyman`. The orientation-resolved table stops at the Lyman limit unless the EUV companion has been computed, and below its first node the dichroic extinction is left at exactly zero — an *omission*, not physics, since a $b/a = 1.4$ spheroid has a nonzero one there. `covers_euv` and `pol_valid_from` are what let a reader tell the two apart. `/kext` of this branch carries `C_polext` and its own `pol_valid_from` for the same reason.

`qjori` stores all three orientations as they are (1: $k \parallel a$; 2: $k \perp a$, $E \parallel a$; 3: $k \perp a$, $E \perp a$), not the dichroic $\frac{1}{2}(Q_3 - Q_2)$ or the random-orientation average the solver forms from them: those are one subtraction away, and storing a derived quantity beside what it is derived from is how the two drift apart. `scatmat_aligned/Z` is chunked one band at a time, because a host reads a band at a time.

13.6 Generating them

```
cd sed
./calc_qtable.x # /grid and /qtable, all three models
./calc_kext.x <model> euv # /kext, for each of the three
./calc_polarized_optics.x # /polarized, into the astrodust product
```

The order matters. `calc_qtable.x` lays down the wavelength axis, so it creates the file with `H5F_ACC_TRUNC` and `/kext` goes with it; the euv run of `calc_kext.x` puts it back. That is the only order in which the two can be consistent, since a curve integrated on the previous axis would be filed against wavelengths it was not computed at. `calc_qtable.x` says so on stdout, and `calc_kext.x` writes `/kext` only if the grid it integrated on equals `/grid/lambda` to 10^{-12}.

13.7 Reading them from Python

`pyutil/sedust_h5.py` is the counterpart of the Fortran reader: same file, same cut, same numbers.

```
from pyutil.sedust_h5 import read_grid, read_kext, read_qtable, describe

k = read_kext('data/zubko/sedust_zubko.h5') # non-ionizing, 866 points
q = read_qtable('data/dl07/sedust_dl07.h5', 'sil', include_euv=True)
describe('data/astrodust/sedust_astrodust.h5') # or: python -m pyutil.sedust_h5 <file>
```

The cut comes from the file's own `i_lyman`, never from a wavelength comparison of the reader's own, so Fortran and Python slice at the same node. Cross sections come back `(n_a, n_lam)`: Fortran declares them `(n_lam, n_a)` and HDF5 stores them in that order, so each grain radius is one contiguous spectrum and the wavelength is the last axis. Nothing is computed on read — a value the file does not carry comes back `None`, never a zero that could be mistaken for a computed one.

13.8 Building without HDF5

`make HDF5=0` (and `HDF5=0 ./build_lib.sh`) compiles the HDF5 paths out. Every entry point of the layer then reports "not available" through its `ok` argument and the callers fall through to the text products, so no consumer carries an `#ifdef` and nothing has to be configured away. The same happens at run time in a tree that simply has no products yet. A default archive references the HDF5 libraries, so a host linking it must put `-lsedust` *before* them: a static archive resolves left to right, and it is `libsedust.a` that needs those symbols.

14. Data files

Model data lives under `SEDust/data/`, in **one directory per model** plus what the models share:

- `astrodust/`, `dl07/`, `zubko/` — everything that model owns: its HDF5 product `sedust_<model>.h5` (§13.), the same optics as text (`q_*.dat`, `kext_*.dat`), and, where the model IS a set of files, its definition. For astrodust that is the T-matrix Q tables, the orientation-resolved `q_DH21Ad_*.dat.gz` with its `DH21_wave` / `DH21_aeff`

/ `DH21_wave_to_12keV` axes, and the aligned scattering matrix `scatmat_aligned_*.dat.gz`; for Zubko, the ZDA config/formula, the tabulated size distributions, the ZDA Q-tables and the calorimetry tables from the Camps et al. 2015 benchmark (a ready descriptor is `zubko_descriptor.txt`).

- `dielectric/` — shared material data: the DH21 astrodust index `index_DH21Ad_P0.20_0.00_1.400` (which the astrodust Q table was computed from, and which fixes how far `lam_min` may reach), the D03 astrosilicate and graphite indices, the D16 turbostratic-graphite tables, and the PAH cross sections. A dielectric function belongs to no one model — DL07 and Zubko read the same D03 astrosilicate — so it does not sit inside a model directory.
- `release/` — the published reference tables and the HD23 size distribution (`size_distribution.dat`), which astrodust and DL07 share.
- SEDust's own regenerated orientation-resolved table stays where the T-matrix driver wrote it, `../tmatrix/output/q_astrodust_jori_*.dat.gz` — it is a sweep output, opt-in through `qpol_path`, and the only one with a birefringence block — as does the EUV companion `q_astrodust_jori_euv_*.dat.gz`, which does *not* ship (§7.4).
- Size-integrated transport optics, written by `calc_kext.x`, see §8. — `../data/astrodust/kext_astrodust_MW.dat` (1129 rows, 0.0912–39810 μm, with the dichroic 8th column), `kext_astrodust_MW_euv.dat` (1762 rows, from 10^{-4} μm), `kext_dl07_MW.dat` (1129 rows, 0.0912–39810 μm), `kext_dl07_MW_euv.dat` (1823 rows, from 6.205×10^{-5} μm), `kext_zubko_BARE_GR_S.dat` (866 rows, 0.08998–10^4 μm) and `kext_zubko_BARE_GR_S_euv.dat` (1201 rows, from 10^{-3} μm). A host that reads a file reads one of these; a host that links the library gets the same curves from `dust_extinction`, which reads one of these files too — the three EUV products are the defaults, see §5.. Each non-EUV product is the row subset of its EUV counterpart that starts at the Lyman limit — same curve, same nodes, over the overlap.

The rule is that a host ships `data/<model>/` for each model it runs, plus `dielectric/` and `release/`. `build_dust` takes `data_dir` and resolves the rest, so nothing names a product file.